Technische Universität Dresden

Fakultät Wirtschaftswissenschaften

Professur für Wirtschaftsinformatik, insbesondere Intelligente Systeme und Dienste

Gysi – Ein konversationeller ML-Assistent:

Entwurf und Implementierung eines chatbasierten Systems zur natürlichsprachlichen Konfiguration von Machine Learning Pipelines

Oliver Hummig

Matrikelnummer: 5011394

Hannes Forhoff

Matrikelnummer: 5016326

Forschungsbeleg

Betreuender Professor

Prof. Dr. Patrick Zschech

Eingereicht am: 27.07.2026

Zusammenfassung

Diese Arbeit untersucht, wie Machine-Learning-Pipelines über natürliche Sprache gesteuert werden können, ohne die Kontrolle über ihre technische Ausführung an ein Large Language Model abzugeben. Hierzu wird mit Gysi ein prototypischer konversationeller ML-Assistent entwickelt. Das Sprachmodell übersetzt Nutzereingaben in strukturierte und validierbare Aktionen, während sämtliche Datenverarbeitungs-, Modellierungs- und Evaluationsschritte deterministisch in einem Python-Backend ausgeführt werden.

Die Pipeline wird versioniert gespeichert und kann über Bestätigungsmechanismen, Warnungen und Rollbacks kontrolliert verändert werden. Der Prototyp unterstützt zentrale Schritte der Datenerkundung, Vorverarbeitung, Modellierung und Ergebnisinterpretation anhand eines Bike-Sharing-Datensatzes.

Ein durchgängiger Showcase und eine explorative Nutzerstudie mit drei Personen zeigen, dass Gysi den technischen Zugang zu ML-Workflows erleichtern kann. Besonders positiv wurden die natürliche Sprachinteraktion, das Kontrollgefühl und die Rückkehr zu früheren Pipeline-Zuständen bewertet. Einschränkungen zeigten sich bei komplexeren fachlichen Entscheidungen und der Interpretation methodischer Zusammenhänge.

Die Arbeit zeigt damit, dass natürliche Sprachsteuerung und reproduzierbare Backend-Ausführung in einem kontrollierten Assistenzsystem verbunden werden können. Gysi ist als prototypischer Machbarkeitsnachweis einzuordnen und stellt kein autonomes AutoML-System dar.

Inhaltsverzeichnis

Zusammenfassung **II**

Abbildungsverzeichnis **VI**

Tabellenverzeichnis **VII**

1 Einleitung **1**

1.1 Motivation 1

1.2 Zielsetzung und Aufbau 2

2 Grundlagen und Problembeschreibung **5**

2.1 Anwendungs- und Datensatzkontext 5

2.2 Definition zentraler Begriffe 6

2.3 Problemstellung und Abgrenzung 8

3 Verwandte Arbeiten **10**

3.1 LLM-basierte Data-Science-Agenten am Beispiel von *Data Interpreter* . . 10

3.2 Natürlichsprachliche Workflow-Erzeugung am Beispiel des *IBM DataStage Assistant* 11

3.3 Einordnung von Gysi im Vergleich zu verwandten Ansätzen 13

4 Konzeption **16**

4.1 Anforderungsanalyse 16

4.2 Systemarchitektur 19

4.2.1 Komponentenüberblick 19

4.2.2 Chat UI 19

4.2.3 Backend API und Interaktionsflüsse 21

4.2.4 Spec Manager und akkumulierte Pipeline 22

4.2.5 Pipeline Executor und ML-Logik 22

4.2.6 Result Summarizer 23

4.3 Rollenverteilung: LLM vs. Backend . . . 23
4.3.1 Rolle des LLM . . . 23
4.3.2 Aktionskatalog und JSON-Vertrag . . . 24
4.3.3 Zweistufiger Bestätigungsfluss als Kontrollmechanismus . . . 25
4.3.4 Berechtigungsgrenzen des LLM . . . 25
4.3.5 Rolle des Backends als Control Flow Owner . . . 26
4.4 Pipeline-Spezifikation als zentrale Schnittstelle . . . 26
4.4.1 Konzept und Funktion . . . 26
4.4.2 Struktur der Spec . . . 26
4.4.3 Akkumulation und Merge-Logik . . . 27
4.4.4 Versionierung . . . 28
4.4.5 Spec als Entkopplungsmechanismus . . . 29
4.5 Persistenzkonzept . . . 29
4.5.1 Hybride Persistenzstrategie . . . 29
4.5.2 SQLite-Datenbankschema . . . 29
4.5.3 Dateisystem-Struktur des Artefakt Stores . . . 31
4.5.4 Rollback und Nachvollziehbarkeit . . . 32

5 Implementierung und Evaluation 33
5.1 Realisierung des Prototyps . . . 33
5.1.1 Technologischer Stack . . . 33
5.1.2 Modellauswahl . . . 34
5.1.3 Realisierter Funktionsumfang . . . 36
5.2 Showcase: End-to-End-Walkthrough am Bike-Datensatz . . . 40
5.2.1 Ziel und Ausgangssituation . . . 40
5.2.2 Orientierung und Datensatzverständnis . . . 40
5.2.3 Aufbau der Preprocessing-Pipeline . . . 41
5.2.4 Demonstration eines Data-Leakage-Risikos . . . 41
5.2.5 Rollback und Aufbau einer korrigierten Pipeline . . . 41
5.2.6 Modelltraining und Ergebnisdarstellung . . . 42
5.2.7 Zentrale Beobachtungen aus dem Walkthrough . . . 42
5.3 Nutzerstudie (explorativ) . . . 44
5.3.1 Zielsetzung und Studiendesign . . . 44
5.3.2 Teilnehmende und Ablauf . . . 45
5.3.3 Aufgaben und Erhebungsmethoden . . . 46
5.3.4 Ergebnisse und Diskussion . . . 47

5.3.5 Limitationen 54

6 Diskussion, Fazit und Ausblick **56**

6.1 Diskussion 56

6.2 Limitationen 57

6.3 Fazit und Ausblick 59

Quellenverzeichnis **62**

Abbildungsverzeichnis

4.1 Systemarchitektur von Gysi 20

5.1 Intelligenz-Latenz-Profil der GPT-4.1-Modellfamilie 35

Tabellenverzeichnis

3.1 Einordnung von Gysi im Vergleich zu verwandten Ansätzen 14

4.1 Funktionale Anforderungen an Gysi 17
4.2 Nicht-funktionale Anforderungen an Gysi 18
4.3 Aktionskatalog der LLM-Backend-Schnittstelle 24
4.4 Felder eines Step-Objekts in der Pipeline-Spezifikation 27
4.5 SQLite-Tabellen des Persistenzmodells 30
4.6 Artefakttypen im Dateisystem 32

5.1 Realisierter Preprocessing-Funktionsumfang 36
5.2 Realisierter Modellierungs-Funktionsumfang 37
5.3 Realisierter Visualisierungs- und Erklärungsumfang 37
5.4 Realisierte Systemfunktionen 38
5.5 Bewusst nicht realisierte Funktionen im aktuellen Prototyp 38
5.6 Im Showcase ausgeführte Schritte 43
5.7 Durchschnittliche aufgabenbezogene Bewertungen 48
5.8 Durchschnittliche Bewertungen der Nachbefragung 52

1 Einleitung

1.1 Motivation

Der Einsatz von Machine Learning hat in den vergangenen Jahren in Forschung, technologischer Entwicklung und praktischen Anwendungsfeldern deutlich an Bedeutung gewonnen. So zeigen bibliometrische und patentstatistische Auswertungen der Fraunhofer-Gesellschaft, dass die Anzahl wissenschaftlicher ML-Publikationen sowie ML-bezogener Patentfamilien im Zeitraum von 2006 bis 2016 beziehungsweise 2015 deutlich angestiegen ist (vgl. Döbel et al. 2018). Ein relevantes Anwendungsszenario ist die Analyse strukturierter und tabellarischer Daten, für die unterschiedliche Verfahren wie Entscheidungsbäume, Random Forests, Gradient Boosting und neuronale Ansätze systematisch untersucht werden (vgl. Grinsztajn, Oyallon & Varoquaux 2022). Gleichzeitig bleibt die praktische Anwendung für Personen ohne tiefgreifende Programmierkenntnisse mit erheblichen Hürden verbunden. Das Erstellen, Konfigurieren und Anpassen einer Machine-Learning-Pipeline erfordert Kenntnisse in Bibliotheken wie pandas oder scikit-learn sowie ein Verständnis für Datenvorverarbeitungsstrategien, Modellparameter und Evaluationskonzepte (vgl. McKinney 2010; Pedregosa et al. 2018; Thornton et al. 2013). Dadurch entstehen Zugangshürden für Anwendergruppen, die zwar fachliche Analysefragen formulieren können, aber nicht über vertiefte Programmier- und Machine-Learning-Kenntnisse verfügen.

Large Language Models (LLMs) bieten durch ihre Fähigkeit, natürlichsprachliche Anweisungen zu verarbeiten, einen vielversprechenden Ansatz, um die Interaktion mit datenanalytischen Werkzeugen zugänglicher zu gestalten. Nutzer könnten Analyseabsichten in eigenen Worten formulieren, sodass ein System daraus geeignete Verarbeitungsschritte ableitet. Arbeiten zu GPT-3 zeigen, dass große Sprachmodelle Aufgaben über natürlichsprachliche Instruktionen und wenige Beispiele bearbeiten können (vgl. Brown et al. 2020). Untersuchungen zu Codex sowie zu Natural-Language-to-Code in Data-Science-Notebooks zeigen zudem, dass LLMs grundsätzlich in der Lage sind, Programmier- und Datenanalyseaufgaben aus natürlichsprachlichen Anforderungen abzuleiten (vgl. Chen et al. 2021; Yin et al. 2023).

Jedoch bringen LLM-basierte Systeme eigene Risiken mit sich. Studien zur neurona-

len Textgenerierung zeigen, dass Modelle Inhalte erzeugen können, die nicht durch die Eingabe gedeckt sind und damit als Halluzinationen gelten (vgl. Maynez et al. 2020). Auch in handlungsorientierten LLM-Systemen können Halluzinationen, Fehlerfortpflanzung und nicht zielführende Aktionen auftreten, wenn Ausgaben nicht ausreichend an externe Informationen oder konkrete Systemgrenzen gebunden sind (vgl. Yao et al. 2023). Werden LLMs zudem direkt zur Codegenerierung oder Codeausführung eingesetzt, entstehen zusätzliche Anforderungen an Korrektheit, Sicherheit und Nachvollziehbarkeit, da generierter Code fehlerhaft sein oder unerwünschte Seiteneffekte verursachen kann (vgl. Chen et al. 2021; Xu, Vasilescu & Neubig 2021).

Diese Belegarbeit setzt genau an dieser Spannung an: Wie lässt sich die Stärke von LLMs für die natürlichsprachliche Interaktion nutzen, ohne die Kontrolle über Ausführung, Korrektheit und Reproduzierbarkeit aufzugeben? Als Antwort wird ein System entwickelt, in dem eine strikte Rollentrennung gilt. Das LLM übernimmt ausschließlich die Aufgaben der Intention-Interpretation und der Orchestrierung: Es übersetzt Nutzereingaben in strukturierte JSON-Aktionen und erklärt Ergebnisse in verständlicher Sprache.Die Datenvorverarbeitung, das Modelltraining, die Metrikberechnung und alle weiteren BerechnungenDie Datenvorverarbeitung, das Modelltraining, die Metrikberechnung und alle weiteren Berechnungen werden deterministisch und ausschließlich im Python-Backend ausgeführt. Das LLM erzeugt keinen ausführbaren Code und hat keinen direkten Zugriff auf die Laufzeitumgebung.

Das Ergebnis ist ein konversationeller ML-Assistent, der demonstriert, dass natürlichsprachliche Steuerung und deterministisch kontrollierte Ausführung kein Widerspruch sein müssen, sofern die Systemgrenzen klar definiert und konsequent eingehalten werden.

1.2 Zielsetzung und Aufbau

Das zentrale Ziel dieser Belegarbeit ist die Konzeption und prototypische Implementierung eines konversationellen Machine-Learning-Assistenten, der es ermöglicht, eine Datenvorverarbeitungs- und Modellierungspipeline ausschließlich über natürliche Sprache zu steuern. Das System trägt den internen Namen Gysi.

Um dieses Ziel zu erreichen, wird eine Architektur entwickelt, in der das Large Language Model ausschließlich als Orchestrierungsschicht fungiert. Es übersetzt Nutzereingaben in strukturierte JSON-Aktionen, die das Python-Backend validiert und deterministisch ausführt. Das LLM erzeugt dabei keinen ausführbaren Code und greift zu keinem Zeitpunkt direkt auf Daten oder die Laufzeitumgebung zu. Diese Rollentrennung ist das konzeptionelle Kernprinzip des Systems und soll demonstrieren, dass natürlichsprachliche Steuerung

und kontrollierte, reproduzierbare Ausführung vereinbar sind.

Ein weiteres Teilziel ist die Versionierung des Systemzustands über eine maschinenlesbare Pipeline-Spezifikation. Jede Änderung an der Pipeline erzeugt eine neue Version, auf die der Nutzer jederzeit zurücksetzen kann. Damit wird exploratives Arbeiten kontrolliert unterstützt, ohne dass Zwischenzustände verloren gehen. Das System soll darüber hinaus nicht nur Ergebnisse produzieren, sondern diese auch verständlich erklären, etwa Feature-Importanzen, Modelleigenschaften oder die Auswirkungen einzelner Vorverarbeitungsschritte. Zu diesem Zweck werden drei interpretierbare Modellklassen unterstützt: die lineare Regression und der Decision Tree aus scikit-learn und die Explainable Boosting Machine aus InterpretML.

Alle entwickelten Komponenten werden an einem konkreten tabellarischen Datensatz erprobt. Der Scope ist dabei bewusst auf diesen einen Datensatz beschränkt, um Tiefe vor Breite zu stellen und den Funktionsumfang vollständig ausarbeiten zu können. Die Arbeit erhebt keinen Anspruch auf ein produktionsreifes System, sondern versteht sich als Prototyp, der die technische Machbarkeit des beschriebenen Ansatzes zeigt und dabei die wesentlichen Herausforderungen, Validierung von LLM-Ausgaben, Guardrails, Versionierung und Erklärbarkeit, in einem funktionsfähigen System adressiert.

Aus dieser Zielsetzung ergibt sich zugleich die Struktur der vorliegenden Arbeit: Der Aufbau folgt der Logik, den behandelten Gegenstand zunächst fachlich und methodisch einzuordnen, anschließend das entwickelte System konzeptionell herzuleiten und schließlich dessen technische Umsetzung sowie Bewertung darzustellen. Auf diese Weise wird der Weg von der Problemstellung über die Architekturentscheidung bis hin zur praktischen Erprobung des Prototyps schrittweise nachvollziehbar gemacht.

Nach der Einleitung werden zunächst die fachlichen und begrifflichen Grundlagen geschaffen. In diesem Teil wird der Anwendungskontext des verwendeten Datensatzes erläutert, es werden die für die Arbeit relevanten Kernbegriffe präzisiert und der methodische Rahmen beschrieben, innerhalb dessen das System entwickelt wurde. Gleichzeitig erfolgt eine bewusste Abgrenzung des Problemraums, um deutlich zu machen, welche Fragestellungen im Fokus der Arbeit stehen und welche Aspekte nicht Gegenstand des Prototyps sind.

Darauf aufbauend werden verwandte Arbeiten betrachtet, um den eigenen Ansatz in den bestehenden Forschungs- und Anwendungskontext einzuordnen. Dabei geht es insbesondere darum, Gemeinsamkeiten und Unterschiede zu bereits existierenden Ansätzen herauszuarbeiten und die spezifische Positionierung des hier entwickelten Systems deutlich zu machen. Dieser Vergleich schafft die Grundlage dafür, die späteren Architektur- und Designentscheidungen nicht isoliert, sondern im größeren fachlichen Zusammenhang

zu verstehen.

Den Schwerpunkt der Arbeit bildet anschließend die Konzeption des Systems. In diesem Teil werden zunächst die Anforderungen herausgearbeitet und daraus die grundlegenden Architekturentscheidungen abgeleitet. Anschließend wird das Zusammenspiel der zentralen Komponenten beschrieben und die Rollenverteilung zwischen LLM und Backend systematisch begründet. Ein besonderer Fokus liegt dabei auf der Frage, wie natürlichsprachliche Interaktion mit einer kontrollierten und reproduzierbaren Ausführung verbunden werden kann. Ebenfalls werden die Pipeline-Spezifikation als zentrale Schnittstelle sowie das Persistenzkonzept behandelt, da beide wesentlich für Nachvollziehbarkeit, Versionierung und Erweiterbarkeit des Systems sind.

Im darauffolgenden Kapitel werden die konzeptionellen Überlegungen in ihre technische Umsetzung überführt. Dort werden der verwendete Technologie-Stack, die wesentlichen Implementierungsentscheidungen und die realisierten Pipeline-Schritte beschrieben. Anschließend wird das System anhand eines durchgängigen Anwendungsszenarios am Bike-Datensatz demonstriert. Ergänzend dazu wird eine explorative Evaluation herangezogen, um erste Aussagen zur praktischen Nutzbarkeit und zum Verhalten des Systems im Einsatz zu ermöglichen.

Den Abschluss bilden Diskussion, Fazit und Ausblick. In diesem Teil werden die erzielten Ergebnisse vor dem Hintergrund der eingangs formulierten Zielsetzung reflektiert, die Grenzen des entwickelten Prototyps offen benannt und mögliche Erweiterungen für zukünftige Arbeiten aufgezeigt.

2 Grundlagen und Problembeschreibung

2.1 Anwendungs- und Datensatzkontext

Als Anwendungsfall dieser Arbeit dient ein Bike-Sharing-Datensatz, der Nutzungsdaten eines öffentlichen Fahrradverleihsystems mit Wetter- und Zeitinformationen verbindet. Er bildet damit ein praxisnahes Beispiel dafür, wie sich ein strukturierter Datensatz mit Methoden des maschinellen Lernens analysieren, vorverarbeiten und modellieren lässt. Der gewählte Kontext eignet sich besonders gut, weil er reale, nachvollziehbare Einflussfaktoren mit einer klaren Vorhersageaufgabe verbindet und damit eine anschauliche Grundlage für die Entwicklung eines konversationellen ML-Assistenten bietet.

Warum sich dieser Kontext für die Arbeit eignet, zeigt sich auch in der Struktur des verwendeten Datensatzes. In der für das Projekt verwendeten Konfiguration liegt der Datensatz als tabellarischer Regressionsdatensatz mit 17.379 Einträgen vor. Die Zielvariable ist die Gesamtzahl der Fahrradausleihungen, die in der Spalte cnt erfasst wird. Als Eingangsmerkmale stehen insbesondere zeitbezogene, saisonale und wetterabhängige Informationen zur Verfügung, darunter etwa Datum, Jahreszeit, Monat, Stunde, Wettersituation, Temperatur, gefühlte Temperatur, Luftfeuchtigkeit und Windgeschwindigkeit. Der Datensatz vereint damit numerische, kategoriale und zeitliche Merkmale in einer Form, wie sie für viele klassische Machine-Learning-Anwendungen auf tabellarischen Daten typisch ist.

Aus dieser Struktur ergeben sich unmittelbar die fachlichen Anforderungen an das entwickelte System. Die Kombination aus zeitbezogenen, kategorialen und numerischen Merkmalen macht unterschiedliche Formen der Vorverarbeitung erforderlich. Zeitinformationen müssen in eine für Modelle nutzbare Form überführt werden, kategoriale Merkmale verlangen geeignete Kodierungsstrategien, und inhaltlich ähnliche Variablen wie Temperatur und gefühlte Temperatur werfen Fragen nach Redundanz und Merkmalsselektion auf. Gleichzeitig liegt mit der Vorhersage der Ausleihzahlen eine klar umrissene Regressionsaufgabe vor, an der sich Modellierung, Evaluation und spätere Ergebnisinterpretation nachvollziehbar demonstrieren lassen. Der Datensatz bündelt damit in kompakter Form

genau jene Anforderungen, auf die der in dieser Arbeit entwickelte Assistent ausgerichtet ist.

Hinzu kommt, dass der Anwendungskontext inhaltlich leicht zugänglich ist. Es ist intuitiv verständlich, dass Wetter, Jahreszeit, Tageszeit oder Feiertage das Ausleihverhalten beeinflussen. Gleichzeitig sind die Zusammenhänge nicht vollkommen trivial, sondern enthalten nichtlineare Effekte und potenzielle Wechselwirkungen. Dadurch eignet sich der Datensatz nicht nur für die Modellierung selbst, sondern auch für die spätere Erklärung von Ergebnissen. Wenn das System beispielsweise Zusammenhänge zwischen Wettersituation und Ausleihzahlen oder zwischen Zeitmerkmalen und Nachfrageverläufen beschreibt, bleiben diese Aussagen auch für fachfremde Leser nachvollziehbar. Das ist für eine Arbeit, die sich mit natürlichsprachlicher Interaktion und erklärbarer Systemunterstützung beschäftigt, ein wesentlicher Vorteil.

Insgesamt bildet der Bike-Sharing-Kontext damit eine passende fachliche Grundlage für die vorliegende Arbeit. Er verbindet Praxisnähe mit einer überschaubaren, aber hinreichend anspruchsvollen Datenstruktur und schafft damit einen geeigneten Rahmen, um die Interaktion zwischen natürlichsprachlicher Steuerung, deterministischer Pipeline-Ausführung und nachvollziehbarer Ergebnisinterpretation systematisch zu untersuchen.

2.2 Definition zentraler Begriffe

Für den weiteren Verlauf der Arbeit werden einige Begriffe in einer klar abgegrenzten, projektspezifischen Bedeutung verwendet. Da das entwickelte System zwischen natürlichsprachlicher Interaktion, formaler Beschreibung von Verarbeitungsschritten und technischer Ausführung vermittelt, ist eine eindeutige Begriffsgrundlage notwendig. Die folgenden Definitionen schaffen daher das begriffliche Fundament für die konzeptionellen und implementierungsbezogenen Kapitel.

Als Machine-Learning-Pipeline wird die geordnete Folge von Verarbeitungsschritten verstanden, die einen Datensatz von seinem Ausgangszustand bis zu einem auswertbaren Ergebnis überführt. Eine solche Pipeline kann Schritte der Datenbereinigung, Merkmalskodierung, Merkmalsselektion, Datenteilung und Modellierung umfassen. Ein einzelner Pipelineschritt bezeichnet dabei eine fachlich abgeschlossene Operation mit eindeutigem Typ und definierten Parametern. Die Pipeline ist in dieser Arbeit nicht nur ein technisches Implementierungsdetail, sondern der zentrale Gegenstand der Interaktion zwischen Nutzer, LLM und Backend.

Die Pipeline-Spezifikation bezeichnet die maschinenlesbare Beschreibung des aktuellen Pipelinezustands. Sie hält fest, welche Schritte in welcher Reihenfolge aktiv sind,

mit welchen Parametern sie konfiguriert wurden und auf welche Zielgröße sich die Ausführung bezieht. Damit fungiert sie als verbindliche Repräsentation des Systemzustands. Änderungen an der Pipeline werden nicht durch direkte Codeanpassungen, sondern durch Veränderungen dieser Spezifikation ausgedrückt. Da die Spezifikation versioniert gespeichert wird, dient sie zugleich als Grundlage für Nachvollziehbarkeit, Wiederholbarkeit und Rücksprünge zu früheren Zuständen.

Eine strukturierte Aktion ist eine formal beschriebene Handlungsanweisung, die aus einer natürlichsprachlichen Nutzereingabe abgeleitet wird. Sie übersetzt den in Freitext formulierten Wunsch des Nutzers in eine Form, die vom Backend geprüft und verarbeitet werden kann. Eine solche Aktion kann beispielsweise das Hinzufügen oder Ändern eines Pipeline-Schritts, die Ausführung einer vorhandenen Spezifikation, die Anfrage von Informationen oder die Rückkehr zu einer früheren Version beschreiben. Entscheidend ist dabei, dass nicht der Freitext selbst ausgeführt wird, sondern nur seine validierte, strukturierte Repräsentation.

In Fällen, in denen eine Nutzereingabe nicht unmittelbar ausgeführt werden soll, arbeitet das System mit einem zweistufigen Vorgehen. Ein Vorschlag bezeichnet dabei eine vom LLM erzeugte, aber noch nicht verbindliche Handlungsempfehlung in strukturierter Form. Erst durch die ausdrückliche Bestätigung des Nutzers wird aus diesem Vorschlag eine auszuführende Operation. Dieses Prinzip ist zentral, weil es die Interaktion natürlichsprachlich hält, ohne die Kontrolle über Systemänderungen an das Sprachmodell abzugeben.

Eine Session bezeichnet den zusammenhängenden Interaktions- und Arbeitskontext eines Nutzers. Innerhalb einer Session entsteht im Verlauf der Interaktion eine Folge versionierter Pipeline-Zustände. Jede relevante Änderung an der Spezifikation erzeugt eine neue Version. Ein Run bezeichnet demgegenüber die konkrete Ausführung einer bestimmten Spezifikation beziehungsweise einer bestimmten Spezifikationsversion. Während die Version also einen Zustand beschreibt, bezeichnet der Run den tatsächlichen Ausführungsvorgang dieses Zustands.

Als Artefakt werden persistent gespeicherte Ergebnisse einer Pipeline-Ausführung bezeichnet. Dazu gehören beispielsweise erzeugte Diagramme, CSV-Exporte, statistische Zusammenfassungen oder Modell-Erklärungen. Artefakte sind von besonderer Bedeutung, weil sie die Brücke zwischen technischer Verarbeitung und menschlicher Interpretation bilden. Sie machen Ergebnisse sichtbar, dokumentierbar und später erneut referenzierbar.

Unter einem Rollback wird die Rückkehr zu einem früheren versionierten Zustand der Pipeline verstanden. Dadurch bleibt exploratives Arbeiten möglich, ohne dass einmal getroffene Konfigurationen verloren gehen. Von deterministischer Ausführung wird in die-

ser Arbeit gesprochen, wenn dieselbe Spezifikation unter gleichen Bedingungen durch das Backend reproduzierbar verarbeitet wird und die resultierenden Kennzahlen nicht durch generative Schätzung, sondern durch tatsächliche Berechnung entstehen. Genau diese Trennung zwischen sprachlicher Steuerung und deterministischer Ausführung ist eine der zentralen Eigenschaften des entwickelten Systems.

Die hier definierten Begriffe bilden die begriffliche Grundlage für die folgenden Kapitel. Sie sind notwendig, um das System nicht nur funktional zu beschreiben, sondern seine Architektur, Interaktionslogik und Problemabgrenzung präzise einordnen zu können.

2.3 Problemstellung und Abgrenzung

Die zentrale Problemstellung dieser Arbeit liegt nicht darin, ob sich der Bike-Sharing-Datensatz grundsätzlich mit Methoden des maschinellen Lernens bearbeiten lässt, sondern wie ein solcher Workflow für Nutzerinnen und Nutzer ohne vertiefte Programmier- und Machine-Learning-Kenntnisse zugänglich gemacht werden kann. Im Projektkontext bedeutet das, dass fachlich sinnvolle Arbeitsschritte, etwa das Entfernen von Duplikaten, der Umgang mit fehlenden Werten, die Aufteilung in Trainings- und Testdaten oder das Training eines Modells, nicht mehr direkt über Code, sondern über natürlichsprachliche Eingaben angestoßen werden sollen. Daraus entsteht die Anforderung, unpräzise oder unvollständige Nutzeraussagen in konkrete, systemseitig ausführbare Operationen zu überführen, ohne dabei fachlich problematische Annahmen stillschweigend zu treffen.

Die Schwierigkeit verschärft sich dadurch, dass die natürlichsprachliche Interpretation in Gysi durch ein LLM unterstützt wird, dessen Ausgaben nicht automatisch mit dem tatsächlich implementierten Funktionsumfang des Systems übereinstimmen. Im Projekt zeigt sich diese Diskrepanz unter anderem daran, dass ein Sprachmodell auch Modelle oder Verarbeitungsschritte vorschlagen kann, die im Backend gar nicht verfügbar sind. Ebenso können fachlich ungünstige Schrittfolgen entstehen, etwa wenn modellrelevante Vorverarbeitung vor einem Train-Test-Split erfolgen würde oder wenn unklare Nutzereingaben zu weitreichenden Systemänderungen führten. Die eigentliche Herausforderung besteht daher darin, sprachliche Flexibilität nur in dem Maße zuzulassen, wie sie durch Validierung, Bestätigung und technische Begrenzung kontrolliert werden kann.

Damit verschiebt sich der Fokus der Arbeit von der reinen Modellierung auf die Gestaltung eines kontrollierten Interaktions- und Ausführungssystems. Gefordert ist nicht nur die Bereitstellung einzelner Pipeline-Schritte, sondern ein konsistenter Mechanismus, der Nutzereingaben prüfbar macht, Änderungen am Pipeline-Zustand nachvollziehbar speichert und Ergebnisse eindeutig einer bestimmten Konfiguration zuordnet. Für den prak-

tischen Einsatz im Projekt ist dies besonders wichtig, weil Nutzer verschiedene Pipeline-Varianten explorativ erproben, Ergebnisse vergleichen und bei Bedarf auf frühere Zustände zurückkehren können sollen, ohne dass Zwischenschritte oder Resultate verloren gehen. Die Problemstellung umfasst damit sowohl die Übersetzung von Intentionen als auch die kontrollierte Verwaltung von Zuständen und Ausführungen.

Die Arbeit ist zugleich bewusst abgegrenzt. Untersucht wird kein allgemeines System für beliebige Datensätze, beliebige Modelle oder frei generierten und ausführbaren Code. Gysi ist vielmehr als spezialisierter Prototyp für einen einzelnen tabellarischen Datensatz konzipiert und unterstützt nur einen fest definierten Katalog an Vorverarbeitungs-, Analyse- und Modellierungsschritten. Nicht im Fokus stehen eine offene AutoML-Optimierung, die dynamische Einbindung externer Verfahren, die Verarbeitung beliebiger Fremddaten oder eine produktionsreife Sprachschnittstelle. Ziel ist stattdessen der Nachweis, dass sich ein konversationeller ML-Assistent unter klaren Guardrails so gestalten lässt, dass natürlichsprachliche Bedienung, reproduzierbare Backend-Ausführung und nachvollziehbare Ergebnisse in einem konsistenten System zusammenwirken.

3 Verwandte Arbeiten

3.1 LLM-basierte Data-Science-Agenten am Beispiel von *Data Interpreter*

Ein für die vorliegende Arbeit besonders relevanter verwandter Ansatz ist *Data Interpreter* von Hong et al. (vgl. Hong et al. 2025). Die Arbeit adressiert die Frage, wie Large Language Models für komplexe Data-Science-Workflows eingesetzt werden können, die nicht nur aus einzelnen Abfragen, sondern aus mehrstufigen und voneinander abhängigen Teilaufgaben bestehen. Die Autoren argumentieren, dass bestehende LLM-basierte Agenten in diesem Kontext an Grenzen stoßen, da Data-Science-Probleme häufig durch nichtlineare Zusammenhänge, rekursive Abhängigkeiten, implizite daten- und logikabhängige Schlussfolgerungen sowie hohe Anforderungen an das Kontextmanagement gekennzeichnet sind (vgl. ebd.).

Als Antwort darauf schlagen Hong et al. ein agentisches System vor, das Data-Science-Aufgaben end-to-end bearbeiten soll. Der Kern des Ansatzes besteht aus mehreren eng verbundenen Komponenten: einer hierarchischen Graphmodellierung zur Repräsentation komplexer Aufgabenstrukturen, einer iterativen Graphverfeinerung mit graphbasierter Speicherung von Kontext- und Zwischenergebnissen sowie einer „programmable node generation“, mit der Teilaufgaben in ausführbare Codeeinheiten überführt werden (vgl. ebd.). Dabei unterscheidet das System zwischen einem Task Graph auf Planungsebene und Action Graphs, die aus ausführbaren Code-Sequenzen beziehungsweise feingranularen Action Nodes bestehen. Ein Graph Executor verarbeitet dabei Rückmeldungen aus der Laufzeitumgebung und trägt über Reflexionsmechanismen zur Verifikation und Anpassung der generierten Schritte bei (vgl. ebd.). Treten während der Ausführung Fehler oder fehlgeschlagene Verifikationen auf, werden zunächst die generierten Ausführungsschritte reflektiert und angepasst. Bei wiederholtem Scheitern kann zudem die Aufgabenstruktur verfeinert werden.

Besonders charakteristisch für *Data Interpreter* ist damit die enge Verzahnung von Planung, Toolauswahl, Codegenerierung, ausführungsgestützter Verifikation und iterativer Fehlerbehebung. Das System versteht sich nicht lediglich als natürlichsprachliche

Schnittstelle zu einem bestehenden Workflow, sondern als aktiver Problemlöser, der Data-Science-Aufgaben weitgehend autonom bearbeitet. Die „programmable node generation“ geht dabei über isolierte Funktionsaufrufe hinaus: Werkzeuge, APIs und generierte Codefragmente werden in kontextabhängige Operationen integriert, sodass auch komplexere Verarbeitungsschritte wie Tool-Chaining oder verschachtelte Toolaufrufe möglich werden (vgl. Hong et al. 2025).

Diese Ausrichtung wird auch in der Evaluation deutlich. Hong et al. testen ihren Ansatz auf mehreren Benchmarks, darunter InfiAgent-DABench und DS-Bench für Datenanalysebeziehungsweise Data-ModellierungsModellierungs-Aufgaben, ein eigener ML-Benchmark mit Kaggle-Datensätzen, MLE-Bench Lite, ein Open-ended-Task-Benchmark sowie Teilmengen des MATH-Datensatzes (vgl. ebd.). Die Autoren berichten dabei je nach Benchmark deutliche Leistungssteigerungen gegenüber mehreren Vergleichssystemen wie AutoGen, XAgent, OpenInterpreter, TaskWeaver und OpenHands (vgl. ebd.). Beispielsweise steigert *Data Interpreter* die Leistung auf InfiAgent-DABench von 75,9 % auf 94,9 %, erreicht im ML-Benchmark einen höheren durchschnittlichen Comprehensive Score als die Vergleichssysteme und erzielt auch bei Open-ended Tasks sowie mathematischen Aufgaben bessere Ergebnisse (vgl. ebd.).

Für die vorliegende Arbeit ist *Data Interpreter* insbesondere deshalb relevant, weil der Ansatz exemplarisch zeigt, wie weit aktuelle LLM-basierte Systeme in Richtung autonomer Bearbeitung komplexer Data-Science-Aufgaben entwickelt werden. Die Arbeit bewegt sich damit im gleichen übergeordneten Problemraum wie Gysi, nämlich der Unterstützung datenanalytischer und modellbezogener Prozesse durch LLM-gestützte Systeme. Zugleich verdeutlicht sie, dass aktuelle Forschungsansätze häufig auf stärkere Agentenautonomie, ausführungsnahe Codegenerierung und iteratives Refinement ausgerichtet sind.

Damit stellt *Data Interpreter* einen aufschlussreichen Referenzpunkt für die Einordnung des in dieser Arbeit entwickelten Systems dar. Während Hong et al. die Fähigkeiten eines weitgehend autonomen Data-Science-Agenten ausloten, verfolgt Gysi einen bewusst kontrollierteren Ansatz. Die Unterschiede zwischen beiden Systemen sowie die spezifische Positionierung von Gysi im Spannungsfeld zwischen natürlicher Sprachsteuerung und kontrollierter Backend-Ausführung werden in Abschnitt 3.3 näher herausgearbeitet.

3.2 Natürlichsprachliche Workflow-Erzeugung am Beispiel des *IBM DataStage Assistant*

Einen weiteren für die vorliegende Arbeit relevanten verwandten Ansatz stellen Gupta et al. mit dem *IBM DataStage Assistant* vor (vgl. Gupta et al. 2026). Im Unterschied zu

agentischen Data-Science-Systemen steht hier nicht die weitgehend autonome Bearbeitung offener Analyseaufgaben im Vordergrund, sondern die Übersetzung natürlichsprachlicher Beschreibungen in ausführbare ETL-Workflows innerhalb eines bestehenden Werkzeugs. Die Autoren greifen damit ein praktisches Problem auf, das in vielen grafischen No-Code-ETL-Umgebungen bestehen bleibt: Zwar lassen sich Workflows ohne klassische Programmierung modellieren, dennoch müssen Nutzende verfügbare Transformationen kennen, passende Operatoren auswählen und zahlreiche Eigenschaften manuell konfigurieren (vgl. Gupta et al. 2026).

Als Antwort darauf beschreiben Gupta et al. ein in IBM DataStage integriertes System, das vollständige mehrstufige ETL-Flows direkt aus natürlichsprachlichen Eingaben erzeugen soll (vgl. ebd.). Der Ansatz umfasst drei eng miteinander verbundene Komponenten: einen *Intent-Based ETL Flow Generator*, einen *Transformer Expression Generator* sowie einen *Document Explorer*. Der Flow Generator übersetzt natürlichsprachliche Anforderungen in geeignete ETL-Stages beziehungsweise Operatoren und deren zugehörige Eigenschaften. Dabei wird zunächst vorhergesagt, welche Stages benötigt werden, bevor in einem zweiten Schritt deren Eigenschaften bestimmt werden. Um ungültige oder nicht ausführbare Vorschläge zu begrenzen, greifen die Autoren auf eine vorgängige Extraktion werkzeugspezifischer Informationen aus der offiziellen Dokumentation, auf In-Context Learning mit Few-Shot-Beispielen sowie auf explizite Validierungsmechanismen zurück (vgl. ebd.).

Eine besondere Rolle spielt der *Transformer Expression Generator*, der natürlichsprachlich beschriebene Transformationslogik in gültige Expressions der proprietären DataStage-Syntax überführt (vgl. ebd.). Hierfür werden Funktionslisten, Signaturen und Few-Shot-Beispiele genutzt, um syntaktisch und semantisch korrekte Ausdrücke zu erzeugen. Anschließend wird das Ergebnis erneut validiert, etwa hinsichtlich zulässiger Funktionen, Argumentanzahl und Typverträglichkeit. Ergänzt wird der Ansatz durch einen *Document Explorer*, der mittels Retrieval-Augmented Generation produktbezogene Dokumentation erschließt und Fragen zu Stages, Funktionen, Eigenschaften oder Nutzungshinweisen in natürlicher Sprache beantwortet (vgl. ebd.). Die Autoren betonen dabei insbesondere die Reduktion von Kontextwechseln, weil Entwurf und dokumentationsgestützte Klärung innerhalb derselben Umgebung erfolgen können.

Die Arbeit ist für den Kontext von Gysi deshalb besonders interessant, weil sie Large Language Models nicht primär als autonome Problemlöser, sondern als natürlichsprachliche Schnittstelle zu einem begrenzten und ausführbaren Workflow-Raum einsetzt. Der *IBM DataStage Assistant* ist damit weniger ein autonomer Agent als vielmehr ein System zur strukturierten Überführung von Sprache in wohldefinierte Operationen eines beste-

henden Datenverarbeitungswerkzeugs. Genau diese Perspektive ist für die Einordnung von Gysi relevant, da auch dort nicht beliebige Codeausführung, sondern die kontrollierte Steuerung eines vordefinierten Pipeline-Raums im Vordergrund steht.

Auch die Evaluation unterstreicht den systemorientierten Charakter des Ansatzes. Gupta et al. berichten für die beste getestete Modellkonfiguration eine Genauigkeit von 96,4 % bei der Vorhersage von ETL-Flows, 87,0 % bei Eigenschaftsvorhersagen und 83,6 % bei Transformer-Ausdrücken (vgl. Gupta et al. 2026). Darüber hinaus verweisen sie auf deutliche Zeitersparnisse in ersten Nutzerstudien, insbesondere für weniger erfahrene Nutzende, sowie auf den Vorteil, dass die entwickelte Funktionalität bereits direkt in IBM DataStage eingebettet und damit produktnah einsetzbar ist (vgl. ebd.).

Für das Related-Work-Kapitel ist dieser Ansatz insbesondere deshalb wertvoll, weil er einen anderen Pol als agentische Systeme wie *Data Interpreter* markiert. Während dort die autonome Bearbeitung komplexer Data-Science-Aufgaben im Vordergrund steht, zeigt der *IBM DataStage Assistant*, wie natürlichsprachliche Interaktion innerhalb eines klar begrenzten und werkzeuggebundenen Workflow-Systems produktiv genutzt werden kann. Er bildet damit einen wichtigen Referenzpunkt für die spätere Einordnung von Gysi zwischen agentischer Offenheit und kontrollierter Workflow-Ausführung.

3.3 Einordnung von Gysi im Vergleich zu verwandten Ansätzen

Die beiden betrachteten Arbeiten markieren zwei unterschiedliche Entwicklungsrichtungen LLM-gestützter Systeme für datenbezogene Workflows. *Data Interpreter* steht exemplarisch für einen agentischen Ansatz, bei dem ein LLM-basiertes System komplexe Data-Science-Aufgaben weitgehend autonom plant, in Teilaufgaben zerlegt, Code generiert, über einen Executor ausführt und bei Bedarf iterativ verfeinert (vgl. Hong et al. 2025). Der *IBM DataStage Assistant* repräsentiert demgegenüber einen stärker begrenzten und werkzeuggebundenen Ansatz, bei dem natürliche Sprache als Schnittstelle zu einem bestehenden ETL-System dient und in ausführbare, aber klar durch das Zielsystem vorstrukturierte Workflows übersetzt wird (vgl. Gupta et al. 2026). Gysi lässt sich zwischen diesen beiden Polen verorten: Es übernimmt die Idee der natürlichsprachlichen Steuerung datenbezogener Prozesse, beschränkt diese jedoch bewusst auf einen kontrollierten ML-Workflow mit validierbaren Systemzuständen und deterministischer Backend-Ausführung.

Tabelle 3.1: Einordnung von Gysi im Vergleich zu verwandten Ansätzen

Dimension	*Data Interpreter*	*IBM DataStage Assistant*	Gysi
Zielbereich	Offene Data-Science- und ML-Aufgaben	Erzeugung ausführbarer ETL-Workflows	Konversationelle Steuerung tabellarischer ML-Pipelines
LLM-Rolle	Agentische Planung, Codegenerierung und iterative Verfeinerung	Übersetzung natürlicher Sprache in Stages, Properties und Expressions	Übersetzung in validierbare JSON-Aktionen sowie Erklärung von Ergebnissen
Ausführung	Codeausführung über einen Graph Executor	Ausführung innerhalb der DataStage-Umgebung	Deterministische Ausführung vordefinierter Schritte im Python-Backend
Autonomie	Hoch; weitgehend autonom	Begrenzt; werkzeug- und domänengebunden	Begrenzt und kontrolliert
Kontrolle	Verifikation, Reflexion und iteratives Refinement	Validierung von Stages, Properties und Transformer Expressions	Schema-Validierung, Bestätigung, Versionierung und Rollback
Einordnung	Agentischer End-to-End-Ansatz	Werkzeugintegrierte NL-zu-Workflow-Erzeugung	Kontrollierter konversationeller ML-Assistent

Im Vergleich zu *Data Interpreter* verzichtet Gysi auf einen hohen Grad agentischer Autonomie. Das LLM übernimmt weder die freie Planung komplexer Problemlösestrategien noch die direkte Generierung und Ausführung von Programmcode. Stattdessen ist seine Rolle auf die Übersetzung von Nutzereingaben in strukturierte JSON-Aktionen sowie auf die sprachliche Erklärung von Ergebnissen begrenzt. Die eigentliche Verarbeitung erfolgt ausschließlich im Python-Backend, das nur einen vordefinierten Katalog implementierter Schritte ausführen kann. Dadurch verschiebt sich der Schwerpunkt von möglichst weitreichender End-to-End-Automatisierung hin zu Kontrollierbarkeit, Reproduzierbarkeit und Nachvollziehbarkeit. Gysi versteht sich folglich nicht als autonomer Data-Science-Agent, sondern als kontrolliertes Assistenzsystem für einen eingegrenzten ML-Kontext.

Im Vergleich zum *IBM DataStage Assistant* weist Gysi dagegen eine stärkere Nähe zum eigentlichen Machine-Learning-Workflow auf. Während der DataStage-Ansatz auf ETL- und Datenintegrationsprozesse ausgerichtet ist, fokussiert Gysi die Konfiguration, Ausführung und Interpretation tabellarischer ML-Pipelines. Neben Vorverarbeitungsschritten und Modelltraining umfasst das System daher auch modellbezogene Auswertungen, Run-Vergleiche, erklärende Ausgaben sowie die Möglichkeit, auf frühere Pipeline-Zustände zurückzugehen. Damit ist Gysi zwar ebenso durch einen begrenzten Aktionsraum charakterisiert wie der DataStage Assistant, erweitert diesen jedoch um Zustandsverwaltung und ML-spezifische Ergebnisinterpretation. Die natürliche Sprache fungiert nicht nur als Ein-

gabe für Workflow-Erzeugung, sondern als Schnittstelle zu einem versionierten, interaktiv veränderbaren Analyseprozess.

Gerade in dieser Kombination liegt die spezifische Positionierung von Gysi. Das System übernimmt weder die weitgehende Offenheit agentischer LLM-Ansätze noch bleibt es auf die Generierung werkzeuggebundener Datenflüsse beschränkt. Stattdessen verbindet es natürlichsprachliche Interaktion mit einem explizit kontrollierten Ausführungsmodell. Zentrale Elemente dieser Architektur sind die Trennung von Sprachmodell und Ausführung, die Validierung aller zustandsverändernden Aktionen, der zweistufige Bestätigungsmechanismus, die versionierte Pipeline-Spezifikation sowie die Möglichkeit von Rollback und Re-Execution. Diese Eigenschaften dienen nicht primär der Maximierung von Autonomie, sondern der Absicherung eines explorativen, aber nachvollziehbaren Arbeitsprozesses.

Aus der Gegenüberstellung wird damit deutlich, dass Gysi einen eigenständigen Beitrag im Spektrum verwandter Ansätze einnimmt. *Data Interpreter* zeigt das Potenzial autonomer LLM-Agenten für komplexe Data-Science-Aufgaben, während der *IBM DataStage Assistant* demonstriert, wie natürliche Sprache produktiv in ein bestehendes Workflow-System integriert werden kann. Gysi knüpft an beide Perspektiven an, kombiniert sie jedoch mit einer bewussten Beschränkung des LLMs und einer stärkeren Betonung von Guardrails, deterministischer Ausführung und versionierter Zustandsverwaltung. Die Arbeit positioniert sich damit weder als offenes AutoML- oder Agentensystem noch als reines ETL-Interface, sondern als kontrollierter konversationeller ML-Assistent für tabellarische Daten.

4 Konzeption

4.1 Anforderungsanalyse

Die Anforderungen an Gysi wurden nicht im Rahmen eines formalen Anforderungserhebungsprozesses dokumentiert, sondern im Projektverlauf iterativ konkretisiert. Als zentrale Ausgangspunkte dienten ein vorbereitendes Exposé sowie eine frühe Abstimmung mit dem betreuenden Professor. Die folgende Anforderungsanalyse rekonstruiert daher die funktionalen und nicht-funktionalen Anforderungen auf Basis dieser Grundlagen, der Konsultationsnotizen und des tatsächlich realisierten Systems. Ziel ist es, die fachliche und technische Stoßrichtung des Prototyps nachvollziehbar zu machen und zugleich seine bewussten Grenzen offenzulegen.

Im Zentrum steht die Idee eines konversationellen ML-Assistenten, der Nutzende bei der Erstellung, Anpassung und Auswertung von Machine–Learning–Pipelines unterstützt. Die Bedienung soll dabei nicht über Programmcode, sondern über natürlichsprachliche Texteingaben erfolgen. Programmierkenntnisse dürfen somit keine Voraussetzung für die Nutzung des Systems sein. Die ursprünglich im Exposé angedachte Sprachsteuerung über Speech-to-Text wurde im Projektverlauf bewusst zurückgestellt. Für den präzisen und häufig parameterbehafteten Charakter von ML-Konfigurationen bietet Texteingabe gegenüber Spracheingabe eine höhere Kontrollierbarkeit und Nachvollziehbarkeit. Gleichzeitig hätte eine robuste Speech-to-Text-Integration den Implementierungsaufwand deutlich erhöht, ohne für den Prototyp einen entsprechenden Mehrwert zu schaffen. Spracheingabe bleibt daher eine mögliche Erweiterung für Folgearbeiten.

Tabelle 4.1: Funktionale Anforderungen an Gysi

ID	Anforderung
FA-1	Gysi soll ML-Pipelines über natürlichsprachliche Texteingaben steuerbar machen. Nutzende sollen Pipeline-Schritte anfragen, ändern und bestätigen können, ohne selbst Programmcode schreiben zu müssen.
FA-2	Das System soll einen Preprocessing-Katalog bereitstellen. Dieser umfasst insbesondere Imputation fehlender Werte, Encoding kategorialer Features, Ausreißerbehandlung, reproduzierbaren Train-Test-Split, Duplikatentfernung, Zeitfeature-Extraktion, Filterung hochkorrelierter Features und Bucketing seltener Kategorien.
FA-3	Das System soll das Training von Decision Trees„ Explainable Boosting Machines und Linear Regression und Linear Regression unterstützen. Für die dreidie drei Modelltypen sollen Metriken auf Trainings- und Testdaten ausgegeben werden, insbesondere RMSE, MAE und R^2, um Overfitting sichtbar zu machen.
FA-4	Feature-Selektion soll manuell, filterbasiert über Korrelationsmetriken sowie modellbasiert über CatBoost Feature Importance möglich sein. Die modellbasierte Selektion soll zweistufig erfolgen: zunächst Analyse, anschließend nutzerbestätigter Drop.
FA-5	Gysi soll erklärende Ausgaben erzeugen, insbesondere Feature-Importance-Plots, globale Modellinterpretationen und vergleichende Modellmetriken für Trainings- und Testdaten.
FA-6	Nutzende sollen zu früheren Pipeline-Zuständen zurückkehren können. Diese History- und Versionierungsfunktion adressiert explizit Anforderungen wie „geh mal zwei Schritte zurück“ und grenzt Gysi von einfachen Chatbot-Systemen ab.

Die funktionalen Anforderungen zeigen, dass Gysi nicht lediglich als Chatoberfläche verstanden wird, sondern als kontrollierte Schnittstelle zu einem fest definierten ML-Workflow. Besonders wichtig ist dabei die Trennung zwischen natürlichsprachlicher Interaktion und technischer Ausführung. Das System soll Vorschläge interpretieren und strukturieren, die eigentliche Verarbeitung jedoch reproduzierbar über definierte Backend-Funktionen durchführen. Dadurch wird vermieden, dass das LLM unkontrolliert Code erzeugt oder ausführt.

Neben den funktionalen Anforderungen wurden mehrere nicht-funktionale Anforderungen abgeleitet, die vor allem die Kontrollierbarkeit und Zuverlässigkeit des Systems betreffen. Diese Anforderungen sind für Gysi besonders relevant, weil das System LLM-basierte Interpretation mit datenverarbeitenden Backend-Prozessen kombiniert. Gerade in diesem Zusammenspiel muss verhindert werden, dass mehrdeutige Nutzereingaben unmittelbar zu irreversiblen Änderungen an Daten oder Pipeline-Zuständen führen.

Tabelle 4.2: Nicht-funktionale Anforderungen an Gysi

ID	Anforderung
NFA-1	Alle Berechnungen sollen reproduzierbar und deterministisch ausführbar sein. Das LLM darf keinen ausführbaren Code erzeugen; die Ausführung liegt ausschließlich beim Backend.
NFA-2	Nutzende bleiben zu jedem Zeitpunkt letztinstanzliche Entscheider. Änderungen an Pipeline, Daten oder Modellierung werden nicht ohne explizite Bestätigung übernommen.
NFA-3	Neue Pipeline-Schritte sollen ergänzt werden können, ohne die grundlegende Schnittstelle zwischen LLM und Backend verändern zu müssen.
NFA-4	Der Funktionsumfang wird bewusst auf den Bike-Sharing-Datensatz der UCI mit 17.379 Einträgen beschränkt, um einen stabilen, nachvollziehbaren und evaluierbaren Prototyp zu gewährleisten.

Aus diesen Anforderungen ergibt sich eine bewusste Abgrenzung des Systems. Gysi ist kein AutoML-System, das eigenständig vollständige Modellierungsentscheidungen trifft oder automatisiert optimale Pipelines sucht. Ebenso ist Gysi kein frei ausführender Code-Executor. Das LLM übernimmt nicht die Rolle einer Laufzeitumgebung, sondern dient als Interpretations- und Vorschlagskomponente innerhalb eines kontrollierten Aktionsraums. Hyperparameter-Optimierung, etwa über Bayesian Search, wurde daher als optionale Erweiterung eingeordnet und im Prototyp nicht umgesetzt. Auch die Idee, das LLM zur Laufzeit externe Bibliotheksdokumentation lesen zu lassen, wurde verworfen. Für den definierten Funktionsumfang erwies sich das im Modell vorhandene kontextuelle ML-Wissen in Kombination mit einem klar abgegrenzten Backend-Aktionskatalog als ausreichend.

Die Anforderungsanalyse führt damit zu drei zentralen Gestaltungsprinzipien für die weitere Konzeption: Erstens muss die natürlichsprachliche Interaktion in strukturierte und validierbare Aktionen übersetzt werden. Zweitens muss jede potenziell zustandsverändernde Aktion über ein Bestätigungsmuster abgesichert werden. Drittens muss die Pipeline selbst als persistierbare und versionierbare Spezifikation modelliert werden, damit Zustände nachvollziehbar, wiederherstellbar und erweiterbar bleiben.

4.2 Systemarchitektur

Das System folgt einem geschichteten, backend-zentrierten Architekturprinzip. Die zentrale Entwurfsentscheidung besteht darin, dass das Backend der einzige ausführende und steuernde Akteur ist. Das Large Language Model (LLM) wird demgegenüber als externe Kognitionsressource verstanden, die ausschließlich auf Anfrage des Backends angesprochen wird. Seine Ausgaben beeinflussen Systemzustand und Ausführung nicht unmittelbar, sondern werden zunächst in ein strukturiertes Aktionsformat überführt und anschließend validiert. Abbildung 4.1 zeigt den vollständigen Komponentenüberblick einschließlich der Kommunikationspfade zwischen den beteiligten Schichten.

4.2.1 Komponentenüberblick

Das System gliedert sich in vier logische Schichten. Die Präsentationsschicht besteht aus der Chat UI auf Basis von React. Die Steuerungsschicht wird durch das Gysi Backend auf Basis von FastAPI gebildet. Die externe Kognitionsressource ist die LLM API, im Prototyp über OpenAI angebunden. Die Persistenzschicht umfasst den SQLite State Store, den Artefakt Store sowie den Dataset Store.

Die Schichten kommunizieren ausschließlich über definierte Schnittstellen. Die Chat UI kennt weder die LLM API noch die interne Datenbankstruktur, sondern spricht ausschließlich über HTTP mit der Backend API. Das LLM erhält ebenfalls keinen direkten Zugriff auf Datensatz oder Dateisystem. Stattdessen übermittelt das Backend einen aufbereiteten Kontext und erwartet eine strukturierte JSON-Antwort. Diese Trennung reduziert Kopplung zwischen den Komponenten und stellt sicher, dass fachliche Entscheidungen, Persistenz und Ausführung nicht in die Präsentationsschicht oder in das LLM verlagert werden. Die Pipeline-Spezifikation (im Folgenden kurz: Spec) bildet dabei das zentrale Kommunikations- und Versionierungsobjekt zwischen den Schichten und wird in Abschnitt 4.4 ausführlich beschrieben.

4.2.2 Chat UI

Die Chat UI ist als reine Präsentationsschicht konzipiert. Sie stellt Nachrichten dar, zeigt Vorschlagskarten mit Step-Parametern an und ermöglicht Nutzenden die Bestätigung oder Ablehnung vorgeschlagener Aktionen. Zusätzlich macht sie den Versionsbaum der Pipeline-Spezifikation navigierbar, sodass frühere Zustände nachvollzogen und bei Bedarf wieder aktiviert werden können.

Die Benutzeroberfläche besitzt keine eigene Geschäftslogik für Validierung, Ausfüh-

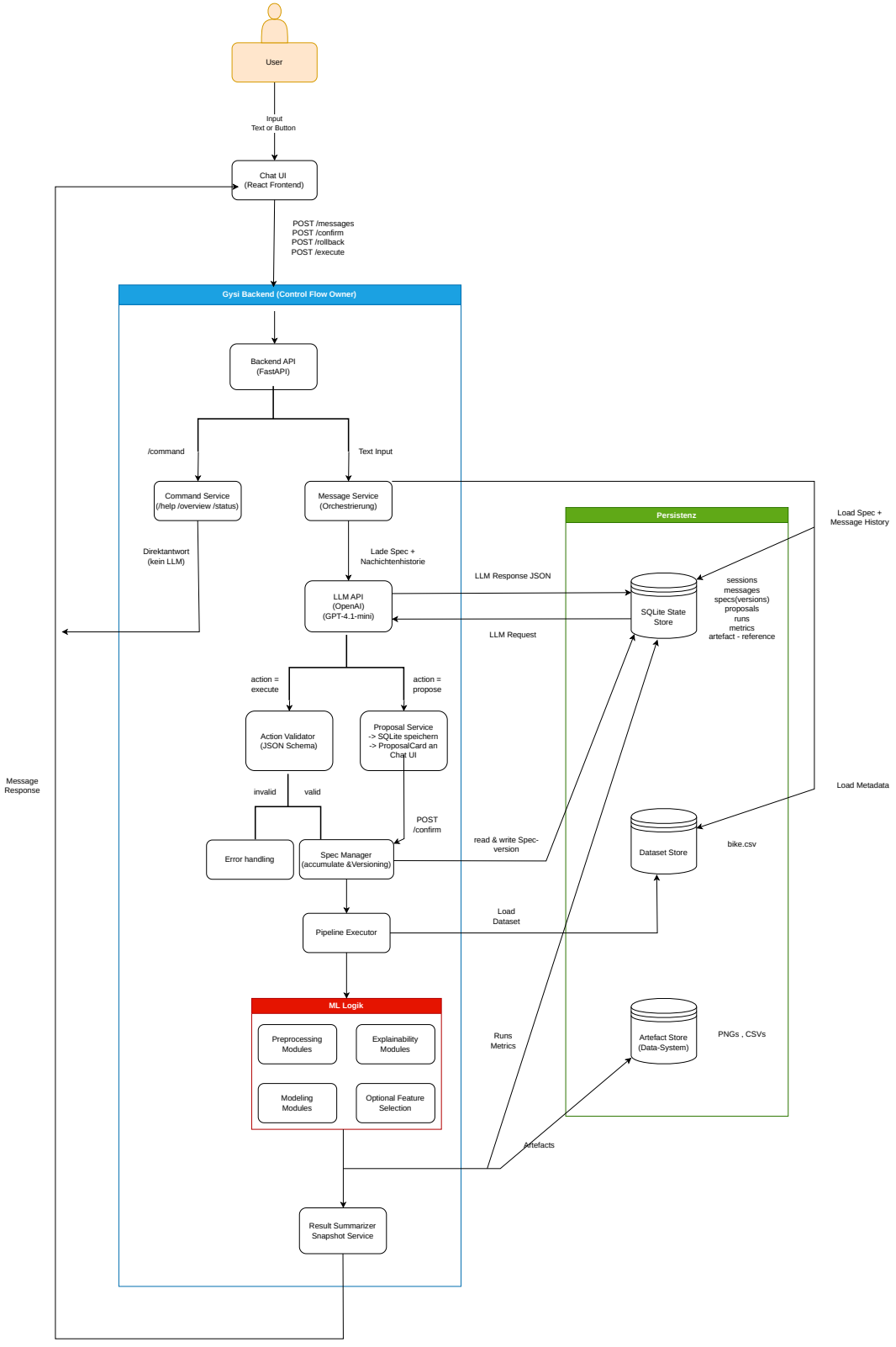


Abbildung 4.1: Systemarchitektur von Gysi

rung oder Zustandsverwaltung. Diese Verantwortlichkeiten liegen vollständig im Backend. Die Kommunikation erfolgt über vier zentrale HTTP-Endpunkte: `POST /messages` für neue Nutzereingaben, `POST /confirm` für die Bestätigung vorgeschlagener Aktionen, `POST /rollback` für das Zurücksetzen auf frühere Spezifikationsversionen und `POST /execute` für die explizite Ausführung einer Pipeline.

4.2.3 Backend API und Interaktionsflüsse

Die Backend API ist der einzige Einstiegspunkt in das System. Sie nimmt Anfragen entgegen, delegiert sie an spezialisierte Services und gibt strukturierte Responses an die Chat UI zurück. Intern lassen sich drei voneinander unabhängige Interaktionsflüsse unterscheiden.

Der erste Interaktionsfluss ist der normale Chat-Flow mit den Aktionen `execute` und `propose`. Eingehende Nutzernachrichten werden vom Message Service verarbeitet. Dieser orchestriert die Anfrage, lädt die aktuelle Spezifikationsversion sowie die Nachrichtenhistorie aus SQLite, liest Dataset-Metadaten wie Spaltennamen und Datentypen aus dem Datensatz und erzeugt daraus einen strukturierten LLM-Kontext. Die Antwort des LLM muss einem definierten JSON-Schema mit den Feldern `action`, `spec` und `explanation` entsprechen. Der Action Validator prüft diese Antwort unmittelbar gegen das Schema und die erlaubten Aktionen. Ungültige Antworten führen zu einer Fehlerantwort, ohne dass Systemzustand oder Pipeline-Ausführung verändert werden.

Das Feld `action` bestimmt anschließend den weiteren Pfad. Bei `execute` führt der Spec Manager die neue Spezifikation mit der bestehenden Session-Spezifikation zusammen und legt eine neue Spezifikationsversion an. Danach führt der Pipeline Executor die Pipeline deterministisch aus. Bei `propose` wird die Spezifikation nicht sofort ausgeführt. Stattdessen speichert der Proposal Service den Vorschlag in SQLite und sendet eine Vorschlagskarte an die UI. Die Ausführung erfolgt erst nach expliziter Bestätigung über `POST /confirm`.

Der zweite Interaktionsfluss ist der Slash-Command-Flow. Befehle wie `/help`, `/status` oder `/overview` werden durch den Command Service erkannt und direkt beantwortet. Eine LLM-Anfrage ist hierfür nicht erforderlich. Dieser Bypass reduziert Latenz und API-Kosten für rein informationsbasierte Anfragen.

Der dritte Interaktionsfluss ist der Rollback-Flow. Über `POST /rollback` setzt das System den aktiven Zustand auf eine frühere Spezifikationsversion zurück. Dazu aktualisiert der Spec Manager den Zeiger auf die aktive Version in SQLite. Eine erneute Ausführung der Pipeline ist davon getrennt und wird nur durchgeführt, wenn Nutzende sie explizit anstoßen.

4.2.4 Spec Manager und akkumulierte Pipeline

Der Spec Manager verantwortet die versionierte Verwaltung der Pipeline-Spezifikation. Ein zentrales Merkmal ist die akkumulierte Session-Spezifikation: Das LLM gibt bei einer Anfrage nicht die vollständige Pipeline zurück, sondern nur neue oder geänderte Schritte. Der Spec Manager führt diese Schritte mit der bestehenden Spezifikation zusammen und erzeugt daraus den neuen gültigen Pipeline-Zustand.

Die Merge-Logik folgt einer typbasierten Regel. Pro Step-Typ existiert in der Regel genau ein Eintrag in der Spezifikation; ein neu übermittelter Step desselben Typs ersetzt den vorhandenen Eintrag. Eine Ausnahme bildet `impute_missing`, da verschiedene Imputationsstrategien wie Median, Mittelwert oder häufigster Wert parallel koexistieren können, sofern sie unterschiedliche Spaltenmengen betreffen.

Jede semantisch wirksame Änderung an der Spezifikation erzeugt eine neue Version mit eindeutiger ID und Elternreferenz in SQLite. Steps, die zwar ausgeführt werden, aber keine persistente Zustandsänderung bewirken, werden als transient klassifiziert und weder in die akkumulierte Spezifikation aufgenommen noch erzeugen sie eine neue Spec-Version. Die genaue Behandlung transienter Steps im Kontext der Versionierung wird in Abschnitt 4.4 beschrieben.

4.2.5 Pipeline Executor und ML-Logik

Der Pipeline Executor ist die einzige Komponente, die Berechnungen auf dem Datensatz ausführt. Er lädt den Bike-Sharing-Datensatz aus dem Dataset Store, iteriert die Steps der aktuellen Spezifikation sequentiell und übergibt den resultierenden DataFrame nach jedem Step an den nächsten. Jeder Step ist als eigenständiges Python-Modul implementiert und folgt einem einheitlichen Interface, das der Executor aufruft.

Vor der Ausführung prüft der Executor die Reihenfolge der Spezifikation auf Data–Leakage–Risiken. Dabei kommen zwei voneinander unabhängige Mechanismen zum Einsatz. Modellierungsschritte wie `train_decision_tree`, `train_ebm` und `catboost_feature_selection` erfordern zwingend einen vorgelagerten `split`-Step. Fehlt dieser, wird die Ausführung mit einem `SpecOrderError` abgebrochen. Lernende Preprocessing-Schritte wie Imputation, kategoriales Encoding, Ausreißerbehandlung und Bucketing seltener Kategorien erzeugen hingegen eine Warnung an die UI, wenn sie vor dem Split-Schritt angeordnet sind. Die Ausführung kann nach expliziter Nutzerbestätigung dennoch fortgesetzt werden, da ein Einsatz auf dem Gesamtdatensatz für explorative oder didaktische Zwecke sinnvoll sein kann.

Nach Abschluss der Ausführung speichert der Executor Run-Metadaten und Metriken

in SQLite. Erzeugte Artefakte wie PNG-Plots oder CSV-Exporte werden im Dateisystem abgelegt. Die zugehörigen Artefaktpfade werden als Referenzen in der Datenbank gespeichert und können über entsprechende Backend-Endpunkte von der UI abgerufen werden.

4.2.6 Result Summarizer

Der Result Summarizer, intern als Snapshot Service umgesetzt, verdichtet die Ergebnisse eines abgeschlossenen Runs zu einem kompakten Kontextobjekt. Dazu liest er Run-Metadaten und Artefaktreferenzen aus SQLite und berechnet Spaltenstatistiken wie Minimum, Maximum, Mittelwert, Anteil fehlender Werte und Histogramminformationen über den resultierenden DataFrame.

Diese Zusammenfassung kann dem LLM für erklärende Antworten bereitgestellt werden. Wie in Abschnitt 4.3 ausgeführt, arbeitet das LLM dabei nicht auf Rohdaten, sondern ausschließlich auf verdichteten Kennzahlen und Artefaktreferenzen. Das Backend bleibt auch hier die kontrollierende Instanz: Es entscheidet, welche Informationen in den LLM-Kontext aufgenommen werden.

4.3 Rollenverteilung: LLM vs. Backend

Ein zentrales Entwurfsprinzip von Gysi ist die strikte Trennung zwischen sprachlicher Interpretation und technischer Ausführung. Diese Trennung ist nicht nur ein Implementierungsdetail, sondern eine grundlegende Architekturentscheidung mit direkten Auswirkungen auf Reproduzierbarkeit, Sicherheit und Wartbarkeit des Systems. Das LLM übernimmt die Rolle eines strukturierten Intentionsübersetzers. Das Backend ist dagegen die einzige ausführende und zustandsverwaltende Instanz.

4.3.1 Rolle des LLM

Das LLM erfüllt im System drei bewusst abgegrenzte Aufgaben. Erstens übernimmt es die Intentionsübersetzung. Dazu empfängt es eine Nutzereingabe in natürlicher Sprache zusammen mit einem strukturierten Systemkontext. Dieser Kontext umfasst die aktuelle Pipeline–Spezifikation, die Nachrichtenhistorie und Dataset–Metadaten. Aus dieser Eingabe leitet das LLM eine formal definierte Aktion ab und gibt sie als JSON-Objekt zurück. Es formuliert keine freien Berechnungsschritte, sondern wählt aus einem vordefinierten Aktionskatalog.

Zweitens erzeugt das LLM erklärende Empfehlungen. Es darf kontextbezogene Hinweise formulieren, etwa warum eine bestimmte Imputationsstrategie sinnvoll ist oder welche Konsequenzen ein Feature-Drop haben kann. Diese Empfehlungen sind jedoch unverbindlich. Sie erscheinen im Feld `explanation` der JSON-Antwort und werden Nutzenden als Text angezeigt. Ausgeführt wird ausschließlich, was das Backend nach erfolgreicher Validierung akzeptiert.

Drittens unterstützt das LLM die Ergebnisinterpretation. Nach einem Pipeline-Run erhält es einen vom Result Summarizer erzeugten Ergebniskontext mit Metriken, Spaltenstatistiken und Artefaktreferenzen. Auf dieser Grundlage kann es die Ergebnisse in natürlicher Sprache erklären. Das LLM arbeitet dabei nicht auf Rohdaten, sondern ausschließlich auf verdichteten Kennzahlen und referenzierten Artefakten.

Die beschriebene Architektur setzt damit eine klar definierte Rollenverteilung zwischen LLM und Backend voraus. Welche Aufgaben dabei explizit dem LLM zugewiesen sind, wo seine Berechtigungsgrenzen liegen und warum das Backend als alleinige ausführende Instanz konzipiert wurde, wird im folgenden Abschnitt begründet.

4.3.2 Aktionskatalog und JSON-Vertrag

Die Kommunikation zwischen LLM und Backend erfolgt über ein formal definiertes JSON-Schema. Jede LLM-Antwort muss die Pflichtfelder `action`, `spec` und `explanation` enthalten. Das Feld `action` ist auf einen festen Wertebereich beschränkt. Tabelle 4.3 fasst die vorgesehenen Aktionen zusammen.

Tabelle 4.3: Aktionskatalog der LLM-Backend-Schnittstelle

Aktion	Semantik
`execute`	Spezifikationsänderung sofort ausführen
`propose`	Spezifikationsänderung vorschlagen und Nutzerbestätigung abwarten
`info`	Reine Informationsantwort ohne Spezifikationsänderung
`rollback`	Rückkehr zu einer früheren Spezifikationsversion
`compare_runs`	Zwei Runs gegenüberstellen
`abort_step_by_step`	Schrittweise Ausführung abbrechen

Dieses Schema wird bei jeder LLM-Antwort durch den Action Validator gegen das JSON-Schema `LLM_RESPONSE_SCHEMA` geprüft. Antwortet das LLM mit einer strukturell ungültigen Ausgabe, etwa freiem Text statt JSON, einem unbekannten Aktionstyp oder

einem fehlenden Pflichtfeld, wird die Antwort verworfen und eine Fehlerantwort an die UI zurückgegeben. Der Systemzustand bleibt in jedem Fall unverändert.

4.3.3 Zweistufiger Bestätigungsfluss als Kontrollmechanismus

Neben der JSON-Schemavalidierung bildet der in Abschnitt 4.2 eingeführte Proposal-Confirm-Flow den zweiten zentralen Schutzmechanismus. Im Folgenden werden die konzeptuellen Eigenschaften dieses Musters vertieft. Wenn das LLM `action: propose` zurückgibt, wird die Spezifikationsänderung nicht ausgeführt. Stattdessen speichert der Proposal Service den Vorschlag mit dem Status `pending` in SQLite und übermittelt eine ProposalCard an die UI. Diese zeigt Nutzenden alle vorgeschlagenen Steps inklusive ihrer Parameter und einer deutschsprachigen Parameterzusammenfassung im Feld `parameters_summary`. Erst nach expliziter Bestätigung über `POST /confirm` wird der Vorschlag durch den Spec Manager übernommen und vom Pipeline Executor ausgeführt.

Dieser Mechanismus gilt auch für potenziell destruktive Analyseschritte. Dies betrifft insbesondere Korrelationsfilter und CatBoost-basierte Feature-Selektion. Beide laufen zunächst im Analysemodus, in dem Ergebnisse angezeigt, aber keine Features entfernt werden. Die finale Entscheidung über Feature-Drops obliegt ausschließlich den Nutzenden. Das LLM darf in beiden Fällen keine eigenständige Selektion vornehmen.

4.3.4 Berechtigungsgrenzen des LLM

Die Berechtigungen des LLM sind durch das Systemdesign und nicht nur durch Konvention eingeschränkt. Es existiert keine technische Möglichkeit für das LLM, ausführbaren Code zu erzeugen, da das Backend ausschließlich JSON-Aktionen und keinen Programmcode akzeptiert. Ebenso kann das LLM keine neuen Pipeline-Bausteine erfinden. Der Aktions- und Step-Katalog ist im Backend über eine Registry, etwa `get_capabilities()`, fest definiert; unbekannte Step-Typen werden vom Executor abgelehnt.

Darüber hinaus besitzt das LLM keinen Dateisystemzugriff und keine direkte Kenntnis über Artefaktinhalte. Bei `propose` kann es außerdem keine automatische Spezifikationsausführung erzwingen, da zwingend eine Nutzerinteraktion erforderlich ist. Schließlich kann es keine Spezifikationsreihenfolge ohne Backend-Validierung durchsetzen. Der Spec-Order-Validator prüft beim Executor, ob Modellierungsschritte, etwa Decision Tree, EBM oder CatBoost Feature Selection, einen vorgelagerten Split-Schritt besitzen. Fehlt dieser, wird die Ausführung mit einem `SpecOrderError` abgebrochen.

4.3.5 Rolle des Backends als Control Flow Owner

Das Backend ist zu keinem Zeitpunkt passiver Empfänger von LLM-Anweisungen. Es bestimmt, wann das LLM angefragt wird, welcher Kontext übergeben wird und ob eine LLM-Antwort den Systemzustand verändern darf. Diese Kontrollinversion – das Backend ruft das LLM auf, nicht umgekehrt – ist der entscheidende Unterschied zu Systemen, in denen ein LLM als autonomer Agent eigenständig Tools aufruft.

In Gysi ist das LLM eine spezialisierte, zustandslose Ressource. Die Zustandsverwaltung, Validierung und Ausführungshoheit liegen vollständig beim Backend. Dadurch bleibt die natürlichsprachliche Flexibilität der Interaktion erhalten, ohne die technische Kontrolle über Daten, Pipeline-Zustände und Ausführung an das LLM zu übertragen.

4.4 Pipeline-Spezifikation als zentrale Schnittstelle

4.4.1 Konzept und Funktion

Die Pipeline-Spezifikation, im Folgenden kurz Spec, ist das zentrale Verbindungsstück zwischen LLM-Orchestrierung und Backend-Ausführung. Sie ist eine maschinenlesbare JSON-Struktur, die vollständig beschreibt, welche Verarbeitungsschritte in welcher Reihenfolge und mit welchen Parametern auf dem Datensatz ausgeführt werden sollen. Das Backend hält die Spec als einzige Quelle der Wahrheit über den aktuellen Pipeline-Zustand. Alle anderen Systemzustände lassen sich daraus ableiten oder rekonstruieren.

Die Spec erfüllt damit eine Doppelfunktion. Einerseits ist sie das Kommunikationsprotokoll zwischen LLM und Backend: Das LLM erzeugt strukturierte Änderungen an der Spec, das Backend validiert und interpretiert sie. Andererseits ist sie das Versionierungsobjekt, über das die History-Funktion des Systems realisiert wird.

4.4.2 Struktur der Spec

Die Spec folgt einem formalen JSON-Schema, dem `PIPELINE_SPEC_SCHEMA`. Auf oberster Ebene enthält sie ein einziges Pflichtfeld: `steps`. Dieses Feld ist ein geordnetes Array von Step-Objekten. Jeder Step besitzt die in Tabelle 4.4 dargestellten Felder.

Tabelle 4.4: Felder eines Step-Objekts in der Pipeline-Spezifikation

Feld	Typ	Bedeutung
`type`	String, Pflichtfeld	Eindeutiger Bezeichner des Step-Typs, etwa `split` oder `train_ebm`
`params`	Object, optional	Step-spezifische Konfigurationsparameter
`enabled`	Boolean, optional	Deaktiviert einen Step, ohne ihn aus der Spec zu entfernen; Standardwert ist `true`

Ein minimales Spec-Beispiel nach einem Split mit anschließender Imputation sieht wie folgt aus:

```
{
  "steps": [
    {
      "type": "split",
      "params": { "test_size": 0.2, "random_state": 42 }
    },
    {
      "type": "impute_missing",
      "params": {
        "strategy": "median",
        "columns": ["windspeed", "hum"]
      }
    }
  ]
}
```

Die Reihenfolge der Steps im Array ist semantisch bedeutsam, da der Executor sie sequentiell ausführt. Das Backend validiert diese Reihenfolge explizit auf Data–Leakage–Risiken, wie in Abschnitt 4.3 beschrieben.

4.4.3 Akkumulation und Merge-Logik

Ein wesentliches Entwurfsprinzip besteht darin, dass das LLM bei jeder Anfrage nur neu hinzukommende oder geänderte Steps zurückgibt, nicht die vollständige Pipeline. Der

Spec Manager führt diese Teilspec mit der bestehenden Session-Spec zusammen. Diese Designentscheidung reduziert die vom LLM zu generierende JSON-Menge und verringert das Risiko, dass bestehende Steps versehentlich überschrieben oder umgeordnet werden.

Die Merge-Logik folgt vier definierten Regeln. Bei der Erstausführung wird die LLM-Spec direkt übernommen, sofern noch keine Session-Spec existiert. Enthält die LLM-Antwort eine leere Step-Liste, also `steps: []`, bleibt die bestehende Spec unverändert und wird erneut ausgeführt. Ein neuer Step-Typ, der noch nicht in der Spec enthalten ist, wird am Ende der Step-Liste angefügt. Eine Ausnahme bildet `cast_dtype`, das stets vor `encode_categorical` eingefügt wird, da Typkonvertierungen vor dem Encoding wirksam sein müssen. Ist ein Step-Typ bereits vorhanden, wird der bestehende Step durch den neuen ersetzt. Spaltenangaben werden dabei spaltenweise zusammengeführt: Listen werden vereinigt und dedupliziert, während Dictionaries, etwa bei `cast_dtype`, Einträge für gleiche Spalten überschreiben.

Die einzige Ausnahme vom Typ-Eindeutigkeitsprinzip ist `impute_missing`. Verschiedene Imputationsstrategien wie Median, Mittelwert oder häufigster Wert können gleichzeitig in der Spec koexistieren, da sie unterschiedliche Spaltenmengen betreffen können. Der Merge-Schlüssel lautet daher nicht nur `impute_missing`, sondern wird um die Strategie erweitert, etwa `impute_missing:median`.

4.4.4 Versionierung

Jede semantisch wirksame Änderung an der Spec erzeugt eine neue Spec-Version in der SQLite-Datenbank. Die Versionsnummern sind linear aufsteigend, beispielsweise `v1`, `v2` und so weiter. Jede Version trägt zudem eine Referenz auf ihre Vorgängerversion, die `parent_spec_id`. Dadurch entsteht ein gerichteter Versionsgraph, der Rollbacks und die Darstellung der Pipeline-Historie ermöglicht.

Vor dem Anlegen einer neuen Version prüft der Spec Manager auf Idempotenz. Ist die neue Spec inhaltlich identisch zur letzten Version, wird keine neue Version erzeugt. Dies verhindert redundante Einträge, insbesondere bei Re-Runs ohne Spezifikationsänderung.

Zusammen mit jeder neuen Version wird eine menschenlesbare `change_summary` gespeichert. Sie wird durch den `spec_diff`-Mechanismus erzeugt und beschreibt kompakt, welche Steps hinzugekommen, geändert oder entfernt wurden. Diese Zusammenfassung wird im Frontend-Versionsbaum angezeigt und macht Änderungen auch ohne direkte Betrachtung der JSON-Struktur nachvollziehbar.

Steps, die keine persistente Zustandsänderung bewirken, werden als transient klassifiziert. Dazu zählt insbesondere `plot_distribution`. Solche Steps werden ausgeführt und ihre Artefakte gespeichert, aber nicht in die akkumulierte Session-Spec aufgenommen.

Entsprechend erzeugen sie auch keine neue Spec-Version.

4.4.5 Spec als Entkopplungsmechanismus

Die Spec entkoppelt LLM-Kommunikation und Backend-Implementierung. Das Backend kann neue Step-Typen hinzufügen, ohne die grundlegende LLM-Schnittstelle anzupassen. Der Capabilities-Endpunkt `GET /capabilities` stellt die Registry aller verfügbaren Steps maschinenlesbar bereit und wird bei jedem LLM-Aufruf als Teil des System-Prompts injiziert.

Das LLM kennt damit den aktuellen Funktionsumfang des Backends, ohne dass Prompt oder Schema manuell gepflegt werden müssen. Gleichzeitig bleibt das Backend die validierende Instanz: Nur Step-Typen und Parameter, die durch Registry und Schema gedeckt sind, können in eine gültige Spec übernommen und anschließend ausgeführt werden. Wie die entstehenden Spec-Versionen, Ausführungsergebnisse und Artefakte dauerhaft gespeichert und verknüpft werden, beschreibt das folgende Persistenzkonzept.

4.5 Persistenzkonzept

4.5.1 Hybride Persistenzstrategie

Das Persistenzkonzept von Gysi folgt einem hybriden Ansatz. Strukturierte und referenzierbare Zustandsdaten werden in einer relationalen SQLite-Datenbank verwaltet, während binäre und dateibasierte Artefakte wie Plots, CSV-Exporte und Parquet-Snapshots im lokalen Dateisystem abgelegt werden. Diese Trennung ist bewusst gewählt: SQLite bietet transaktionale Sicherheit und effiziente Abfragen für Metadaten, Sessions und Versionsgraphen; das Dateisystem erlaubt direkten und performanten Zugriff auf größere Dateien ohne Datenbank-Overhead.

Der Dataset Store, im Prototyp die Rohdatei `bike.csv`, ist statisch und read-only. Er wird durch das System nicht verändert. Der Pipeline Executor liest den Datensatz bei jeder Ausführung neu ein und erzeugt daraus abgeleitete Zwischenstände, Ergebnisse und Artefakte.

4.5.2 SQLite-Datenbankschema

Die SQLite-Datenbank bildet den vollständigen Systemzustand einer oder mehrerer Sessions ab. Sie besteht aus sieben zentralen Tabellen: `sessions`, `spec_versions`, `runs`, `artifacts`, `messages`, `proposals` und `step_states`. Zusammen modellieren diese Ta-

bellen Sitzungen, Pipeline-Versionen, Ausführungen, Artefakte, Gesprächshistorie, offene Vorschläge und den schrittweisen Ausführungsmodus.

Tabelle 4.5: SQLite-Tabellen des Persistenzmodells

Tabelle	Funktion
`sessions`	Wurzeltabelle für Nutzersitzungen mit Statusverwaltung
`spec_versions`	Versionierte Pipeline-Spezifikationen inklusive Elternreferenz und Änderungszusammenfassung
`runs`	Pipeline-Ausführungen mit Status, Zeitpunkten, Metriken und Spec-Bezug
`artifacts`	Metadaten und Dateipfad-Referenzen auf erzeugte Artefakte
`messages`	Gesprächshistorie pro Session für LLM-Kontext und Nachvollziehbarkeit
`proposals`	Noch nicht bestätigte, bestätigte oder abgelehnte LLM-Vorschläge
`step_states`	Fortschritt des schrittweisen Ausführungsmodus

Tabelle `sessions`. Die Tabelle `sessions` ist die zentrale Wurzeltabelle. Jede Nutzersitzung erhält eine eindeutige UUID als `session_id`. Das Feld `status` unterscheidet zwischen aktiven und geschlossenen Sessions. Die zentralen Spalten sind `session_id` als Primärschlüssel, ein optionaler Anzeigename `name`, der Erstellzeitpunkt `created_at` und der Sitzungsstatus `status` mit den Werten `active` oder `closed`.

Tabelle `spec_versions`. Die Tabelle `spec_versions` ist das Kernstück der Versionierung. Jede semantisch wirksame Spec-Änderung erzeugt einen neuen Eintrag. Neben `spec_id` und `session_id` speichert sie die lineare Versionsnummer `version`, die vollständige Spezifikation als `spec_json`, eine optionale Vorgängerreferenz `parent_spec_id`, eine lesbare `change_summary` sowie den Erstellzeitpunkt `created_at`. Die Elternreferenz ermöglicht die Rekonstruktion des vollständigen Versionsgraphen.

Tabelle `runs`. Jede Ausführung der Pipeline wird als Run gespeichert. Ein Run ist stets mit einer Spec-Version verknüpft. Dadurch ist nachvollziehbar, welche Pipeline-Konfiguration welche Ergebnisse produziert hat. DieseDiese speichert Identifikatoren für Run, Session und Spec, die angezeigte Spec-Version, den Ausführungsstatus, Start- und

Endzeitpunkt sowie eine optionale Fehlermeldung. Die Ergebnisspalte enthält verdichtete Ausführungsmetriken wie RMSE, MAE und R^2 als JSON-String.

Tabelle `artifacts`. Artefakte werden nicht als BLOBs in der Datenbank gespeichert, sondern als Dateipfad-Referenzen. Der eigentliche Dateiinhalt liegt im Dateisystem; SQLite hält nur die Referenz und Metadaten. Die Tabelle enthält `artifact_id`, `run_id`, den Artefakttyp `type`, den Dateipfad `file_path` und den Erstellzeitpunkt `created_at`. Über `artifact_id` sind Artefakte über den API-Endpunkt `GET /artifacts/{artifact_id}` abrufbar.

Tabelle `messages`. Die Tabelle `messages` speichert die vollständige Gesprächshistorie pro Session. Jede Nutzer- und Assistenznachricht wird über `message_id`, `session_id`, `role`, `content` und `created_at` abgelegt. Diese History wird bei LLM-Aufrufen als Kontext übergeben, um kohärente Mehrrunden-Konversationen zu ermöglichen.

Tabelle `proposals`. Die Tabelle `proposals` verwaltet offene LLM-Vorschläge, die noch nicht bestätigt wurden. Ein Proposal besitzt den Status `pending`, bis Nutzende es bestätigen oder ablehnen. Die vorgeschlagene Spec-Änderung und die vom LLM generierte Begründung werden gemeinsam mit Proposal-ID, Session-ID, Status und Erstellzeitpunkt gespeichert.

Tabelle `step_states`. Die Tabelle `step_states` verwaltet den schrittweisen Ausführungsmodus. Dieser Modus erlaubt die sequentielle Ausführung und Bestätigung einzelner Steps innerhalb eines Proposals. Der aktuelle Fortschritt wird über JSON-Arrays für ausstehende, abgeschlossene und bestätigte Steps gespeichert. Ergänzend werden State-ID, Session-ID, Proposal-ID und der letzte Aktualisierungszeitpunkt abgelegt.

4.5.3 Dateisystem-Struktur des Artefakt Stores

Artefakte werden unter einem konfigurierbaren Basispfad, im Prototyp `artifacts`, abgelegt. Die Benennung folgt dem Schema `{run_id}_{artefakttyp}.{extension}`. Dadurch sind Artefakte bereits anhand ihres Dateinamens eindeutig einem Run und einem Artefakttyp zuordenbar. Die Datenbank speichert zusätzlich die formale Referenz über die Tabelle `artifacts`.

Tabelle 4.6: Artefakttypen im Dateisystem

Typ	Format	Erzeugt durch
`split_train` / `split_test`	CSV	`split`-Step
`feature_importance`	PNG	`train_decision_tree`
`ebm_global_explanation`	PNG	`train_ebm`
`overfitting_plot`	PNG	Modellierungs-Steps
`learning_curve`	PNG	Modellierungs-Steps
`correlation_filter_result`	CSV	`correlation_filter`
`plot_distribution`	PNG	`plot_distribution`
`column_stats`	JSON	Snapshot Service nach jedem Run
`_snapshot`	Parquet	Snapshot Service, DataFrame-Stand

4.5.4 Rollback und Nachvollziehbarkeit

Die Kombination aus versionierter Spec-Tabelle und run-gekoppelten Artefakten ermöglicht vollständige Nachvollziehbarkeit. Zu jeder Spec-Version existieren die zugehörigen Runs, zu jedem Run die erzeugten Artefakte und Metriken. Damit kann rekonstruiert werden, welche Pipeline-Konfiguration zu welchen Ergebnissen geführt hat.

Ein Rollback auf eine frühere Spec-Version setzt lediglich den Verweis auf die aktive Spec in der Datenbank zurück. Die Artefakte und Runs aller Zwischenversionen bleiben erhalten und sind weiterhin abrufbar. Eine Re-Ausführung der zurückgesetzten Spec ist optional und wird nur durchgeführt, wenn Nutzende sie explizit initiieren.

5 Implementierung und Evaluation

5.1 Realisierung des Prototyps

5.1.1 Technologischer Stack

Ein übergreifendes Leitprinzip bei der Technologiewahl war die lokale Installierbarkeit des gesamten Systems ohne tiefgreifende technische Vorkenntnisse. Der Prototyp soll von Testpersonen, Interessierten und dem betreuenden Professor auf eigenen Rechnern gestartet werden können, ohne dass ein Datenbankserver eingerichtet, ein Cloud-Dienst konfiguriert oder ein lokales Sprachmodell installiert werden muss. Dieses Kriterium hat die Wahl der eingesetzten Technologien maßgeblich beeinflusst.

Das Backend wurde mit Python und FastAPI umgesetzt. FastAPI war im Projektteam bereits bekannt, wodurch Einarbeitungszeit reduziert und ein vertrautes Entwicklungsumfeld sichergestellt werden konnte. Darüber hinaus bietet FastAPI mehrere technische Eigenschaften, die sich direkt mit den Anforderungen des Systems decken. Die integrierte Pydantic-Validierung erzwingt Typsicherheit an den API-Grenzen und unterstützt damit NFA-1, die Forderung nach reproduzierbarem und deterministischem Verhalten. Routen und Schemas werden automatisch als interaktive OpenAPI-Dokumentation bereitgestellt, was das Debugging während der Entwicklung erheblich vereinfachte. Für den lokalen Betrieb genügt ein einzelner `uvicorn`-Prozess; ein dedizierter Application-Server ist nicht erforderlich.

Als Persistenzlösung wurde SQLite gewählt. Die Datenbank liegt als einzelne Datei im Projektverzeichnis und benötigt keinen laufenden Datenbankserver. Für Testpersonen entsteht dadurch kein zusätzlicher Einrichtungsaufwand. Trotz dieses geringen Gewichts ist SQLite vollständig ACID-konform und damit für transaktionale Schreiboperationen, etwa das Anlegen neuer Spec-Versionen oder das Speichern von Run-Metadaten, zuverlässig geeignet. Für den Umfang eines lokalen Einzelnutzer-Prototyps mit einer begrenzten Anzahl an Sessions wäre ein schwergewichtiges Datenbanksystem wie PostgreSQL funktional überdimensioniert und würde den Installationsaufwand unnötig erhöhen.

Das Frontend wurde als React-Anwendung umgesetzt. React war im Team bekannt und

erlaubt eine klare komponentenbasierte Strukturierung der Oberfläche, wobei Nachrichten, Vorschlagskarten und der Versionsbaum als eigenständige Komponenten abgebildet werden. Der Start der Anwendung erfordert lediglich `npm install` und `npm start`; ein zusätzliches Frontend-Setup ist nicht notwendig.

5.1.2 Modellauswahl

Das Large Language Model übernimmt in Gysi keine eigenständige Reasoning- oder Coding-Aufgabe, sondern fungiert als Orchestrator eines fest definierten Python-Backends. Es empfängt strukturierten Kontext, wählt aus einem vorgegebenen Aktionskatalog und gibt ein JSON-Objekt zurück, das einem formalisierten Schema entsprechen muss. Für diesen Einsatzbereich sind robuste Instruktionsbefolgung, zuverlässige Ausgabe in vorgegebenen Formaten, ein ausreichend großes Kontextfenster und niedrige Latenz die entscheidenden Kriterien. Komplexe mehrstufige Schlussfolgerungen sind dagegen nicht erforderlich.

Lokale Open-Source-Modelle wurden als Alternative erwogen, jedoch verworfen. Ihr Betrieb setzt leistungsfähige Hardware sowie technisches Vorwissen zur Modellinstallation voraus und widerspricht damit dem Leitprinzip der einfachen lokalen Testbarkeit. Ein Tester ohne ML-Erfahrung sollte den Prototyp nutzen können, ohne zunächst eine eigene Inferenzinfrastruktur einzurichten.

Für den Einsatz über die OpenAI-API wurde GPT-4.1 mini ausgewählt. Das Modell wird in der offiziellen Dokumentation als kleinere und schnellere Variante von GPT-4.1 beschrieben, die insbesondere für Instruktionsbefolgung und Tool-Calling geeignet ist (vgl. OpenAI 2025a). Mit einem Kontextfenster von mehr als einer Million Tokens können Systemprompt, Nachrichtenhistorie, aktuelle Pipeline-Spezifikation und verdichtete Ausführungsergebnisse gemeinsam verarbeitet werden, ohne dass Kontext frühzeitig verworfen werden muss (vgl. ebd.).

Abbildung 5.1 zeigt das Intelligenz-Latenz-Profil der GPT-4.1-Familie auf Basis der offiziellen Modelldokumentation (vgl. OpenAI 2025c). GPT-4.1 mini positioniert sich dabei als leistungsfähiges Mittelfeld-Modell mit geringerer Latenz als größere Modelle der Familie. OpenAI beschreibt GPT-4.1 mini zudem als Modell, das GPT-4o in mehreren Benchmarks erreicht oder übertrifft und zugleich Latenz und Kosten deutlich reduziert (vgl. OpenAI 2025b).

Die verfügbaren Benchmarks bestätigen die Eignung für den Orchestrator-Einsatz. Auf MMLU erreicht GPT-4.1 mini 87,5 % und weist damit eine breite fachliche Abdeckung auf. Für die Instruktionsbefolgung sind zwei Benchmarks besonders relevant: IFEval, das streng vorgegebene Ausgabeformate bewertet, liefert 84,1 % und ist damit für eine An-

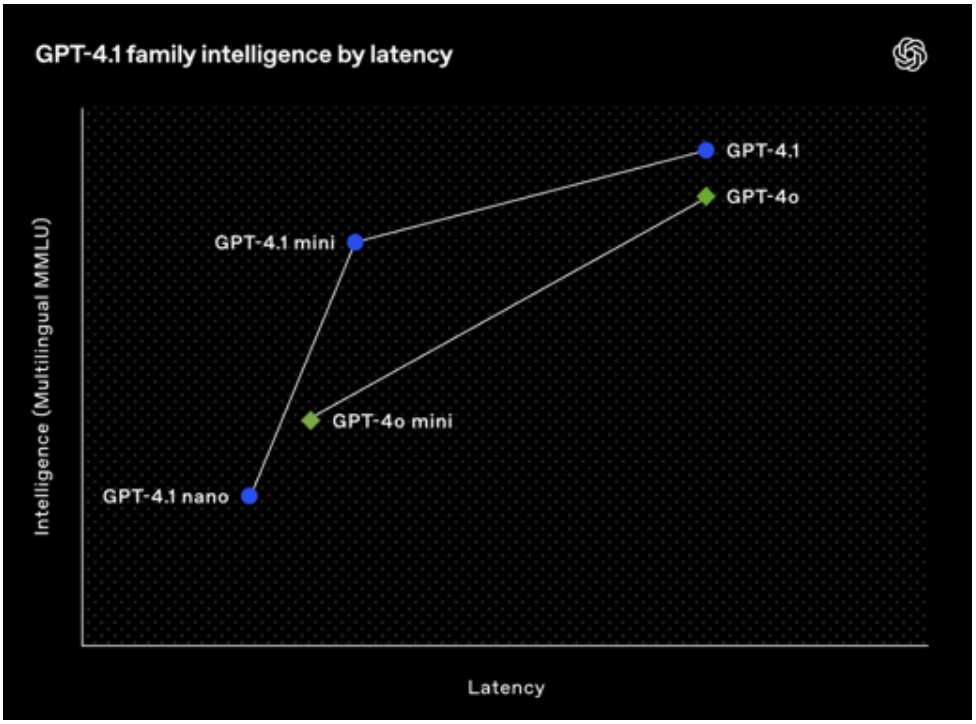


Abbildung 5.1: Intelligenz-Latenz-Profil der GPT-4.1-Modellfamilie

wendung relevant, die ausschließlich valides JSON als Ausgabe akzeptiert. ComplexFuncBench misst die robuste Nutzung von Funktionsaufrufen mit geeigneten Parametern; GPT-4.1 mini erreicht hier 49,3 %, was dem Einsatz als Orchestrator des Backend-Step-Katalogs entspricht. Der MultiChallenge-Wert von 35,8 % zeigt zudem, dass das Modell auch über längere Dialogverläufe hinweg mehrteilige Aufgaben bearbeiten kann (vgl. OpenAI 2025b).

Neben der Leistungsfähigkeit war die Kostenseite für den Prototyp entscheidend. GPT-4.1 mini wird mit 0,40 US-Dollar pro einer Million Input-Tokens und 1,60 US-Dollar pro einer Million Output-Tokens abgerechnet (vgl. OpenAI 2025a). Selbst bei intensiver Nutzung mit vielen iterativen Dialogen und unterschiedlichen Pipeline-Konfigurationen bleiben die Gesamtkosten damit überschaubar. Da die Projektkosten von den Autoren selbst getragen wurden, war dieser Aspekt bei der Modellauswahl ausschlaggebend.

Insgesamt bietet GPT-4.1 mini damit das Profil, das Gysi benötigt: instruktionsstabil, formatkonform, mit großem Kontextfenster und niedrigen Betriebskosten sowie über eine verwaltete API ohne eigene Inferenzinfrastruktur nutzbar.

5.1.3 Realisierter Funktionsumfang

Tabelle 5.1, Tabelle 5.2, Tabelle 5.3 und Tabelle 5.4 fassen den im Prototyp realisierten Funktionsumfang zusammen. Die Zuordnung orientiert sich an den in Abschnitt 4.1 definierten funktionalen Anforderungen. Bewusste Abgrenzungen sind in Tabelle 5.5 aufgeführt.

Tabelle 5.1: Realisierter Preprocessing-Funktionsumfang

Schritt	**Funktion und zentrale Parameter**
`dedup`	Entfernt identische Duplikate. Keine Parameter.
`drop_columns`	Entfernt explizit benannte Features. Parameter: `columns`. Die Zielvariable `cnt` ist geschützt und nicht löschbar.
`cast_dtype`	Konvertiert Datentypen vor nachgelagerten Schritten. Parameter: `columns`, optional `datetime_format`.
`impute_missing`	Imputation fehlender Werte. Parameter: `strategy` (`median`, `mean`, `most_frequent`), `columns`.
`outlier_handling`	Ausreißerbehandlung per IQR oder Z-Score. Parameter: `method`, `zscore_threshold`, `columns`.
`encode_categorical`	Kodierung kategorialer Merkmale. Parameter: `method` (`label`, `onehot`), `columns`.
`rare_category_bucketing`	Bündelt seltene Kategorien. Parameter: `min_frequency`, `columns`.
`time_feature_encoding`	Extrahiert Zeitmerkmale aus Datumsfeldern. Parameter: `column`, `features`, `drop_original`.
`correlation_filter`	Identifiziert und entfernt hoch korrelierte Features. Parameter: `threshold`, `analysis_only`, `features_to_drop`.
`split`	Reproduzierbarer Train-Test-Split. Parameter: `test_size`, `random_state`.

Tabelle 5.2: Realisierter Modellierungs-Funktionsumfang

Schritt	Funktion und zentrale Parameter
`train_linear_regression`	Trainiert ein lineares Regressionsmodell. Keine Modellparameter im Step. Ausgabe von Train- und Testmetriken (`RMSE`, `MAE`, R^2).
`train_decision_tree`	Trainiert einen Decision Tree Regressor. Parameter: `max_depth`, `random_state`. Ausgabe von Train- und Testmetriken.
`train_ebm`	Trainiert eine Explainable Boosting Machine. Parameter: `random_state`. Ausgabe von Train- und Testmetriken.
`catboost_feature_selection`	Feature-Selektion über CatBoost Importance. Parameter: `top_n`, `analysis_only`, `features_to_drop`, `random_state`. Zweistufiger Analyse- und Bestätigungsmodus.

Tabelle 5.3: Realisierter Visualisierungs- und Erklärungsumfang

Ausgabe	Typ	Quelle
`plot_distribution`	Step-basierte Visualisierung	Eigener Visualisierungsschritt mit Parametern `feature` und optional `group_by`.
`feature_importance`	PNG	Automatisch aus `train_decision_tree`.
`validation_curve`	PNG	Automatisch aus `train_decision_tree`, `train_ebm` und `train_LR`.
`ebm_global_explanation`	PNG	Automatisch aus `train_ebm`.
`lr_coefficients`	PNG	Automatisch aus `train_linear_regression`.
`column_stats`	JSON	Snapshot Service nach jedem Run.
`snapshot`	Parquet	Snapshot Service nach jedem Run.
`predicted_vs_actual`	PNG	Automatisch nach jedem Modell Run.
`distribution_actual_vs_predict`	PNG	Automatisch nach jedem Modell Run.
`residual_distribution`	PNG	Automatisch nach jedem Modell Run.

Tabelle 5.4: Realisierte Systemfunktionen

Funktion	**Umsetzung im Prototyp**
Proposal-Confirm-Flow	Zweistufige Ausführung mit expliziter Nutzerbestätigung vor zustandsändernden Aktionen.
Spec-Versionierung	Versionierte Pipeline-Zustände mit Historie, Elternreferenz und Änderungszusammenfassung.
Rollback	Rücksetzen auf frühere Spec-Versionen über dedizierten API-Endpunkt.
Slash-Commands	`/help`, `/overview`, `/status` ohne LLM-Anfrage über Command Service.
Erweitertes Hilfesystem	Konfigurierbare Hilfetexte in `help_texts.yaml` mit Kategorien und schrittbezogener Hilfe.
Analysemodus	`analysis_only` für `correlation_filter` und `catboost_feature_selection`.
Data–Leakage–Absicherung	Harte Split-Pflicht für Modellierungsschritte sowie Warnmechanismus für lernende Preprocessing-Schritte vor dem Split.
Schema- und Aktionsvalidierung	Prüfung von LLM-Antworten gegen `LLM_RESPONSE_SCHEMA` und Step Registry.

Tabelle 5.5: Bewusst nicht realisierte Funktionen im aktuellen Prototyp

Funktion	**Status**
Speech-to-Text	Nicht umgesetzt. Fokus auf textbasierter Interaktion.
Automatisierte Hyperparameter-Optimierung	Nicht umgesetzt. Als optionale Erweiterung vorgesehen.
Laufzeitzugriff des LLM auf externe Dokumentation	Nicht umgesetzt. Im Scope durch festen Aktionskatalog ersetzt.

Damit wird der Funktionsumfang des Prototyps vollständig und nachvollziehbar dokumentiert. Gleichzeitig bleibt die Abgrenzung zum geplanten, aber nicht realisierten Zielumfang transparent.

5.2 Showcase: End-to-End-Walkthrough am Bike-Datensatz

5.2.1 Ziel und Ausgangssituation

Dieses Kapitel demonstriert das entwickelte System anhand eines vollständigen Anwendungsszenarios auf dem Bike-Sharing-Datensatz. Ziel des Walkthroughs ist es, die Interaktionslogik von Gysi nicht isoliert über Einzelfunktionen, sondern als zusammenhängenden Arbeitsablauf sichtbar zu machen. Gezeigt wird, wie ein Nutzer den Datensatz zunächst exploriert, anschließend eine Preprocessing-Pipeline über natürlichsprachliche Eingaben aufbaut, ein Modell trainiert und die resultierenden Ausgaben interpretiert. Der Showcase dient damit zugleich als funktionaler Nachweis der zuvor beschriebenen Architekturprinzipien, insbesondere der natürlichsprachlichen Steuerung, der strukturierten Aktionsübersetzung, der deterministischen Backend-Ausführung und der versionierten Zustandsverwaltung.

Zu Beginn wird eine neue Session im Frontend erzeugt. Bereits in dieser Phase zeigt sich ein zentrales Merkmal des Systems: Gysi unterscheidet zwischen rein informierenden und tatsächlich zustandsverändernden Anfragen. Dadurch wird der Walkthrough nicht nur als Modellierungsbeispiel, sondern auch als Demonstration der Interaktionslogik des Systems relevant.

5.2.2 Orientierung und Datensatzverständnis

Im ersten Schritt verschafft sich der Nutzer einen Überblick über den Datensatz. Über den Slash-Command /overview kann ohne LLM-Ausführung eine kompakte Übersicht über den Bike-Datensatz abgerufen werden. Das Backend liefert dabei Informationen zu Spaltennamen, Datentypen, fehlenden Werten und grundlegenden Verteilungen. Ergänzend dazu kann im Daten-Tab der Rohdatensatz betrachtet werden, einschließlich paginierter Zeilenansicht und statistischer Spalteninformationen.

Auf diese Weise erhält der Nutzer einen ersten Eindruck vom Datenbestand, bevor eigentliche Pipeline-Schritte ausgelöst werden. Für den Showcase ist dies wichtig, weil hier bereits sichtbar wird, dass nicht jede natürlichsprachliche Eingabe automatisch in eine zustandsverändernde Operation überführt wird. Gysi kann den Datensatz somit zunächst erklären und zugänglich machen, bevor aktive Verarbeitungsschritte angestoßen werden.

5.2.3 Aufbau der Preprocessing-Pipeline

Im nächsten Schritt beginnt der eigentliche Aufbau der Pipeline. Anders als in klassischen Notebook-Workflows werden die Verarbeitungsschritte nicht direkt im Code formuliert, sondern als natürliche Sprache an den Assistenten gerichtet. Im Showcase steht dabei zunächst die Behandlung fehlender Werte im Vordergrund. Eine typische erste Eingabe besteht darin, fehlende Werte in der Spalte windspeed mit dem Median zu imputieren. Das System übersetzt diese Anfrage in eine strukturierte Aktion, validiert die Parameter gegen den verfügbaren Schrittkatalog und führt anschließend den entsprechenden `impute_missing`-Step im Backend aus.

Auf dieser Grundlage wird die Pipeline schrittweise erweitert. Im weiteren Verlauf kann beispielsweise One-Hot-Encoding für kategoriale Variablen wie season und weathersit angefordert werden. Danach kann ein Ausreißer-Handling für numerische Spalten wie hum und windspeed folgen. Jeder dieser Schritte erzeugt eine neue Pipeline-Version, die im Frontend nachvollziehbar dargestellt wird. Gleichzeitig bleiben frühere Zustände erhalten, sodass nicht nur die aktuelle Pipeline, sondern auch deren Entwicklungspfad sichtbar wird. Damit wird deutlich, dass Gysi nicht nur einzelne Operationen ausführt, sondern einen persistenten, explorativ veränderbaren Arbeitskontext verwaltet.

5.2.4 Demonstration eines Data-Leakage-Risikos

Ein besonders wichtiger Teil des Walkthroughs ist die Demonstration der Reihenfolgeabhängigkeit von Pipeline-Schritten. Wird die Imputation vor dem Train-Test-Split durchgeführt, basiert die Schätzung fehlender Werte auf dem gesamten Datensatz und damit auch auf Informationen aus den späteren Testdaten. Diese Reihenfolge ist aus methodischer Sicht problematisch, weil sie zu Data–Leakage führen kann.

Gysi macht diese Problematik sichtbar, indem das System beim anschließenden Versuch, ein Modell zu trainieren, einen Warnhinweis ausgibt. Auf diese Weise zeigt der Showcase nicht nur, dass Pipeline-Schritte technisch ausgeführt werden können, sondern auch, dass das System ihre Anordnung auf typische Fehlerquellen hin überprüft. Die Verantwortung verbleibt zwar beim Nutzer, die Oberfläche unterstützt ihn jedoch dabei, statistisch fragwürdige Konfigurationen zu erkennen.

5.2.5 Rollback und Aufbau einer korrigierten Pipeline

Im weiteren Verlauf des Walkthroughs wird deshalb auf einen früheren Pipeline-Zustand zurückgegangen und eine methodisch saubere Variante aufgebaut. Der Nutzer setzt den

Zustand auf eine frühere Version zurück und führt den Train-Test-Split nun vor den lernenden Vorverarbeitungsschritten aus. Anschließend wird die Imputation erneut angewendet, diesmal auf Basis der Trainingsdaten.

Durch dieses Vorgehen zeigt der Showcase exemplarisch, wie Gysi exploratives Arbeiten unterstützt. Frühere Varianten gehen nicht verloren, sondern bleiben im Versionsbaum erhalten, während neue Pfade als alternative Entwicklungslinien aufgebaut werden können. Die Möglichkeit zum Rollback ist dabei nicht nur ein Komfortmerkmal, sondern ein zentrales Element zur kontrollierten Interaktion mit Pipeline-Zuständen.

5.2.6 Modelltraining und Ergebnisdarstellung

Nach dem Aufbau der korrigierten Pipeline wird ein Modelltraining durchgeführt, etwa mit einem Decision Tree. Das System führt die zuvor spezifizierten Schritte deterministisch im Backend aus und stellt die Ergebnisse anschließend im Frontend bereit. Im Info-Bereich werden die erzeugten Metriken für Trainings- und Testdaten angezeigt, insbesondere RMSE, MAE und R^2. Darüber hinaus können erzeugte Artefakte wie Feature-Importance-Plots oder weitere Modelloutputs direkt eingesehen werden.

Dadurch wird deutlich, dass Gysi den Nutzer nicht nur bis zur Ausführung eines Modells begleitet, sondern auch bei der Interpretation der Resultate unterstützt. Die natürlichsprachliche Oberfläche dient somit nicht allein der Pipeline-Konfiguration, sondern ebenso der Ergebniserschließung.

5.2.7 Zentrale Beobachtungen aus dem Walkthrough

Der End-to-End-Walkthrough macht insgesamt sichtbar, dass Gysi weder als reiner Chatbot noch als offenes agentisches Codegenerierungssystem konzipiert ist. Vielmehr handelt es sich um ein kontrolliertes Assistenzsystem, in dem natürlichsprachliche Eingaben in strukturierte, überprüfbare und versionierbare Pipeline-Operationen überführt werden.

Gerade im Zusammenspiel aus explorativer Datensatzsicht, schrittweiser Pipeline-Erstellung, Warnung vor problematischen Konfigurationen, Rollback und nachvollziehbarer Ergebnisdarstellung zeigt sich der praktische Mehrwert der gewählten Architektur. Der Showcase erfüllt damit eine doppelte Funktion: Er demonstriert die Benutzbarkeit des Systems aus Nutzersicht und belegt zugleich, dass die zuvor entwickelten Entwurfsprinzipien im prototypischen System tatsächlich umgesetzt wurden.

Tabelle 5.6: Im Showcase ausgeführte Schritte

Bereich	Schritt	Beschreibung
Exploration	`/overview`	Abruf einer kompakten Übersicht über Spalten, Datentypen, fehlende Werte und Verteilungen ohne Ausführung eines Pipeline-Schritts.
	Datenansicht	Anzeige des Rohdatensatzes im Frontend mit paginierter Tabellenansicht und statistischen Spalteninformationen.
Preprocessing	`impute_missing`	Imputation fehlender Werte in `windspeed` mit der Median-Strategie als erster persistenter Pipeline-Schritt.
	`encode_categorical`	One-Hot-Encoding für kategoriale Variablen wie `season` und `weathersit`.
	`outlier_handling`	Behandlung von Ausreißern in numerischen Spalten wie `hum` und `windspeed`.
Modeling	Leakage-Demonstration	Imputation vor dem Train-Test-Split, anschließend Trainingsversuch zur Demonstration eines methodisch problematischen Workflows.
	Rollback	Rückkehr zu einer früheren Pipeline-Version, um eine korrigierte Variante aufzubauen.
	`split`	Aufteilung des Datensatzes in Trainings- und Testdaten vor lernenden Vorverarbeitungsschritten.
	`train_decision_tree`	Training eines Decision-Tree-Modells auf der korrigierten Pipeline.

5.3 Nutzerstudie (explorativ)

Ob Gysi sein zentrales Versprechen einlöst, Nutzer:innen ohne Programmier- oder ML-Kenntnisse einen nachvollziehbaren Zugang zu ML-Workflows zu ermöglichen, lässt sich nicht allein durch technische Prüfungen beantworten. Hierzu wurde ergänzend eine Nutzerstudie durchgeführt, deren Zielsetzung und Design im Folgenden beschrieben werden.

5.3.1 Zielsetzung und Studiendesign

Die Studie untersucht, wie verständlich, vertrauenswürdig und bedienbar Gysi beim tatsächlichen Bearbeiten von Aufgaben erlebt wird. Ziel ist es, die subjektive Verständlichkeit, Bedienbarkeit und das Kontrollerleben der Nutzer:innen zu erfassen und daraus qualitative Hinweise auf Stärken und Schwächen des Prototyps zu gewinnen. Daraus ergeben sich folgende Leitfragen:

- Können Nutzer:innen ohne Programmier- oder ML-Vorkenntnisse mit Hilfe von Gysi einen typischen Ablauf von der Datenexploration über die Behandlung fehlender Werte und weitere Vorverarbeitungsschritte bis zum Modelltraining eigenständig nachvollziehen und durchführen?
- Sind die Rückmeldungen, Erklärungen und Warnungen des Systems verständlich genug, um informierte Entscheidungen zu treffen?
- Behalten Nutzer:innen das Gefühl, jederzeit Kontrolle über Änderungen am Datensatz zu haben?
- Wie wird die natürlichsprachliche Bedienung im Vergleich zu klassischen, code- oder GUI-basierten Werkzeugen erlebt?

Diese Fragen sind bewusst formativ statt summativ ausgerichtet: Es geht nicht darum, Gysi gegen ein Referenzsystem zu benchmarken oder statistisch abgesicherte Aussagen über eine Grundgesamtheit zu treffen, sondern darum, aus der Perspektive realer Nutzer:innen konkrete Verbesserungspotenziale und Verständnisprobleme zu identifizieren.

Entsprechend dieser formativen Zielsetzung wurde die Studie als explorative, aufgabenbasierte Nutzerstudie mit kombiniert qualitativem und quantitativem Erhebungsansatz konzipiert. Auf ein experimentelles Design mit Kontrollgruppe oder Vergleichssystem wurde bewusst verzichtet: Bei einem Prototyp im Rahmen einer Belegarbeit steht nicht der statistische Nachweis eines Effekts im Vordergrund, sondern das Verständnis, wie einzelne Nutzer:innen mit dem System interagieren und wo Verständnis- oder Vertrauensbrüche auftreten.

Den Kern der Studie bildet ein Aufgabenblatt mit sechs aufeinander aufbauenden Aufgaben, die einen typischen ML-Workflow von der Datenexploration bis zum Modelltraining abdecken und jeweils durch offene Leitfragen sowie kurze Einschätzungsskalen begleitet werden; Aufgaben und Erhebungsinstrumente im Detail folgen in Abschnitt 5.3.3. Die Aufgaben sind bewusst offen formuliert und geben keinen konkreten Wortlaut für die Eingabe vor, sodass die Teilnehmenden ihre Anliegen in eigener, natürlicher Sprache formulieren müssen. Das entspricht der realen Nutzungssituation und erhöht die ökologische Validität der Studie gegenüber einem stark standardisierten Skript.

Die Kombination aus aufgabenbegleitender qualitativer Reflexion und einer zusammenfassenden quantitativen Skala erlaubt es, trotz der für eine Belegarbeit typischen kleinen Stichprobe sowohl konkrete, aufgabenbezogene Beobachtungen als auch eine grobe, vergleichbare Gesamttendenz festzuhalten. Die quantitativen Werte werden dabei nicht als statistisch belastbare Kennzahlen interpretiert, sondern als deskriptive Ergänzung zu den qualitativen Aussagen der Teilnehmenden.

5.3.2 Teilnehmende und Ablauf

An der Studie nahmen insgesamt drei Proband:innen teil. Einziges Auswahlkriterium bei der Rekrutierung war das Fehlen von Vorkenntnissen in Machine Learning oder Programmierung, da genau diese Personengruppe die eigentliche Zielgruppe von Gysi darstellt: Nutzer:innen, die ML-Workflows über natürliche Sprache statt über Code oder klassische GUI-Tools umsetzen sollen. Die Stichprobe ist damit klein und nicht repräsentativ, für eine explorative, formativ ausgerichtete Studie im Rahmen einer Belegarbeit jedoch ausreichend, um wiederkehrende Verständnis- und Bedienprobleme sichtbar zu machen.

Die Sitzungen fanden einzeln statt, jeweils begleitet von mindestens mindestens einer Person„ die den Ablauf beobachtete, ohne selbst in die Bearbeitung einzugreifen. Zu Beginn jeder Sitzung wurde kurz erläutert, dass es sich um einen Prototypen und kein fertiges System handelt. Anschließend bearbeitete jede Testperson das Aufgabenblatt eigenständig und in fester Reihenfolge von Aufgabe 1 bis Aufgabe 6 im direkten Dialog mit Gysi, wobei alle drei Personen dieselben Aufgaben in derselben Reihenfolge erhielten, um die Ergebnisse trotz der geringen Stichprobengröße untereinander vergleichbar zu halten. Die Sitzung endete jeweils mit den offenen Abschlussfragen und dem standardisierten Nachbefragungsbogen.

5.3.3 Aufgaben und Erhebungsmethoden

Das Aufgabenblatt umfasst sechs Aufgaben, die aufeinander aufbauen und in ihrer Summe einen vollständigen, typischen Ablauf mit Gysi nachbilden:

1. **Datensatz erkunden**: ersten Überblick über Spalten, Inhalte und Auffälligkeiten des Datensatzes gewinnen.
2. **Fehlende Werte verstehen und behandeln**: fehlende Werte (z. B. in `windspeed`) erkennen, mögliche Strategien verstehen und eine davon ausführen.
3. **Weitere Vorverarbeitungsschritte erkennen**: zusätzliche sinnvolle Schritte prüfen und mindestens einen umsetzen.
4. **Modelltraining und Ergebnisverständnis**: ein Modell trainieren und die gelieferten Kennzahlen grob einordnen.
5. **Hinweise, Warnungen und Rückmeldungen interpretieren**: Warnungen des Systems beobachten, verstehen und eine Reaktion ableiten.
6. **Versionierung, Verlauf und Variantenbildung**: zu einem früheren Zustand zurückkehren und darauf aufbauend eine alternative Entscheidung testen.

Jede Aufgabe folgt demselben Aufbau aus Ausgangslage, Ziel, Arbeitsauftrag und einem Hinweis, der klarstellt, dass keine fachlich perfekte Lösung erwartet wird, sondern die eigene Nachvollziehbarkeit im Vordergrund steht.

Zur Erhebung werden pro Aufgabe zwei Instrumente kombiniert. Erstens beantworten die Teilnehmenden vier aufgabenspezifische, offene Leitfragen im Freitext (z. B. welche Auffälligkeiten erkannt oder welche Strategie gewählt wurde und warum). Zweitens schätzen sie direkt im Anschluss ihre Wahrnehmung auf drei fünfstufigen Skalen ein: Verständlichkeit der Systemunterstützung, Sicherheit bei der eigenen Entscheidung und Gefühl der Orientierung im System. Beide Instrumente werden unmittelbar nach der jeweiligen Aufgabe erhoben, um Verzerrungen durch nachträgliche Erinnerung zu vermeiden.

Nach Abschluss aller sechs Aufgaben folgen sieben offene Abschlussfragen zur retrospektiven Gesamteinschätzung, etwa dazu, bei welcher Aufgabe die Unterstützung am stärksten bzw. am schwächsten empfunden wurde, ob eigene Entscheidungen möglich waren oder das Gefühl vorherrschte, dem System zu folgen, und welche konkrete Verbesserung am wichtigsten wäre.

Den Abschluss bildet ein standardisierter Nachbefragungsbogen mit zehnstufigen Zustimmungsskalen (1 = stimme überhaupt nicht zu, 10 = stimme vollständig zu), gegliedert in vier Dimensionen mit je drei Items: *Verständlichkeit* (Nachvollziehbarkeit der

Systemhandlungen, Verständlichkeit der Rückmeldungen, Verständlichkeit der Warnungen), *Bedienbarkeit* (Ausdruck eigener Absichten in natürlicher Sprache, Intuitivität der Bedienung, Klarheit über die nächsten Schritte), *Kontrolle* (Kontrollgefühl über Änderungen, Nutzen der Bestätigungsmechanismen, Nutzen der Rücksprungmöglichkeit) sowie *Gesamteindruck* (Verständlichkeit des Systems insgesamt, Nutzbarkeit ohne Programmierkenntnisse, erleichterter Zugang zu ML-Workflows). Diese vier Dimensionen liefern eine kompakte, über alle Teilnehmenden vergleichbare Zusammenfassung, die die aufgabenbezogenen qualitativen Befunde ergänzt.

5.3.4 Ergebnisse und Diskussion

Die Auswertung kombiniert die aufgabenbezogenen Skalenbewertungen mit den Freitextantworten der Teilnehmenden. Die quantitativen Angaben werden ausschließlich deskriptiv verwendet. Aufgrund der Stichprobengröße von drei Personen erlauben die Mittelwerte keine statistisch belastbaren Aussagen. Sie dienen dazu, aufgabenbezogene Tendenzen sichtbar zu machen und anhand der qualitativen Rückmeldungen einzuordnen.

Für jede der sechs Aufgaben bewerteten die Teilnehmenden die Verständlichkeit der Systemunterstützung, die Sicherheit bei der eigenen Entscheidung und das Gefühl der Orientierung auf einer Skala von 1 bis 5. Für sämtliche Aufgaben liegen Bewertungen aller drei Teilnehmenden vor. In einem Erhebungsbogen wurden bei der Verständlichkeit von Aufgabe 4 sowohl der niedrigste als auch der höchste Skalenwert markiert. Da die zugehörige Freitextantwort die Erklärungen des Systems als grundsätzlich verständlich und teilweise nachvollziehbar beschreibt, wurde die Markierung am positiven Skalenende als intendierte Bewertung gewertet. Diese Zuordnung ist bei der Interpretation des entsprechenden Mittelwerts zu berücksichtigen.

Tabelle 5.7: Durchschnittliche aufgabenbezogene Bewertungen

Aufgabe	Verständlichkeit	Entscheidungssicherheit	Orientierung
1: Datensatz erkunden	4,33	3,67	3,67
2: Fehlende Werte behandeln	4,67	4,33	4,33
3: Weitere Vorverarbeitung	3,33	4,67	4,00
4: Modelltraining und Ergebnisse	4,67	4,00	4,67
5: Hinweise und Warnungen	4,67	4,67	4,33
6: Versionierung und Variantenbildung	4,33	4,33	3,67

Anmerkung: Bewertet wurde auf einer Skala von 1 bis 5. Höhere Werte entsprechen einer höheren wahrgenommenen Verständlichkeit, Entscheidungssicherheit beziehungsweise Orientierung. Allen Mittelwerten liegen drei Bewertungen zugrunde.

Datensatzerkundung. Der Einstieg in den Datensatz wurde insgesamt positiv bewertet. Die durchschnittliche Verständlichkeit der Systemunterstützung lag bei 4,33. Die Teilnehmenden konnten den Gegenstand des Datensatzes erfassen, relevante Spalten identifizieren und Unsicherheiten zu einzelnen Variablen in natürlicher Sprache formulieren. Die Freitextantworten zeigen, dass Gysi insbesondere bei der Erklärung zunächst nicht selbsterklärender Spaltenbezeichnungen unterstützte. Auch erste Zusammenhänge zwischen zeitlichen, meteorologischen und nachfragebezogenen Variablen wurden erkannt.

Die niedrigeren Werte für Entscheidungssicherheit und Orientierung von jeweils 3,67 zeigen jedoch, dass ein verständlicher Systemdialog nicht automatisch zu einer vollständigen Orientierung im Anwendungskontext führte. Genannt wurden eine unübersichtliche Aufzählung der vorhandenen Spalten sowie eine fehlende initiale Einordnung des Datensatzes und des Analyseziels. Der Datensatz konnte mithilfe gezielter Rückfragen erschlossen werden, war jedoch nicht in allen Fällen bereits durch die initiale Darstellung ausreichend verständlich. Für die Zielsetzung bedeutet dies, dass Gysi den fachlichen Einstieg erleichtert, eine strukturierte Einführung in Datensatz, Zielvariable und möglichen Arbeitsablauf aber nicht vollständig ersetzt.

Behandlung fehlender Werte. Aufgabe 2 erreichte mit 4,67 die höchste Verständlichkeitsbewertung unter den ersten drei Aufgaben. Alle Teilnehmenden konnten die betroffenen Spalten erkennen, mögliche Behandlungsstrategien erfragen und eine konkrete Stra-

tegie auswählen. In allen Durchgängen wurde die Medianimputation als geeignete Option identifiziert. Die Freitextantworten zeigen, dass diese Entscheidung mit ihrer Robustheit, dem vergleichsweise geringen Aufwand oder der begrenzten Beeinflussung der Werteverteilung begründet werden konnte.

Die Werte für Entscheidungssicherheit und Orientierung von jeweils 4,33 sprechen dafür, dass eine klar abgegrenzte Vorverarbeitungsentscheidung über natürliche Sprache grundsätzlich nachvollziehbar getroffen werden konnte. Einschränkungen zeigten sich bei den methodischen Konsequenzen der Ausführungsreihenfolge. Insbesondere die Bedeutung einer Imputation vor oder nach der Trainings- - und Testaufteilung war nicht allen Teilnehmenden unmittelbar klar. In einem Durchgang wurde eine informationsbezogene Nachfrage zudem direkt als Ausführungsabsicht interpretiert. Die Aufgabe bestätigt damit die grundsätzliche Zugänglichkeit der Systeminteraktion, zeigt aber zugleich, dass Erklärungen zu Auswirkungen und Reihenfolge vor der Ausführung erforderlich sind.

Weitere Vorverarbeitung. Aufgabe 3 weist mit 3,33 den niedrigsten Verständlichkeitswert der Aufgabenstudie auf. Das System stellte unterschiedliche mögliche Verarbeitungsschritte bereit, darunter die Ableitung von Zeitmerkmalen, die Kodierung kategorialer Variablen, die Behandlung von Ausreißern, das Entfernen von Duplikaten und Verfahren zur Merkmalsselektion. Die Freitextantworten zeigen jedoch, dass mehrere dieser Schritte ohne zusätzliche Erläuterungen nicht vollständig verstanden wurden.

Auffällig ist die gleichzeitig hohe Entscheidungssicherheit von 4,67. Diese Kombination kann nicht als uneingeschränkt positives Ergebnis interpretiert werden. Die qualitativen Angaben legen nahe, dass einzelne Entscheidungen auf Grundlage der Systemempfehlung getroffen wurden, obwohl die fachlichen Konsequenzen nur teilweise nachvollzogen werden konnten. In einem Durchgang wurden mehrere Schritte unmittelbar ausgeführt, obwohl zunächst eine Einordnung der sinnvollsten Optionen erfragt worden war. In einem weiteren Durchgang wurden nur diejenigen Schritte ausgewählt, die ohne zusätzliche Erklärung verständlich erschienen.

Die Aufgabe verdeutlicht damit den Unterschied zwischen technischer Ausführbarkeit und informierter Entscheidungsfähigkeit. Gysi ermöglichte die Konfiguration weiterer Verarbeitungsschritte ohne Programmcode. Die Unterstützung reichte jedoch nicht durchgehend aus, um Alternativen gegeneinander abzuwägen und die Auswirkungen auf den späteren Modellierungsprozess eigenständig zu beurteilen. Dieser Befund begrenzt die Zielerreichung insbesondere bei komplexeren und weniger klar abgegrenzten Entscheidungen.

Modelltraining und Ergebnisverständnis. Das Modelltraining wurde hinsichtlich Verständlichkeit und Orientierung mit jeweils 4,67 positiv bewertet. Alle Teilnehmenden konnten ein Modell über natürliche Sprache anstoßen. In den dokumentierten Durchgängen wurden insbesondere Decision Tree Regressoren trainiert. Das System stellte Modellkennzahlen bereit und erklärte deren Bedeutung in natürlicher Sprache.

Die Freitextantworten zeigen, dass Kennzahlen wie RMSE, MAE und R^2 grundsätzlich erfasst und grob eingeordnet werden konnten. In einem Durchgang wurde zudem ein mögliches Overfitting aufgrund eines unbegrenzten Entscheidungsbaums erkannt. Eine weitere Person beschrieb die Erläuterungen der Ergebnisse als ausführlich und klar. Damit unterstützte Gysi nicht nur die technische Ausführung des Modelltrainings, sondern auch eine erste Interpretation des Resultats.

Die Entscheidungssicherheit fiel mit 4,00 etwas geringer aus. Ohne Referenzwerte oder kontextbezogene Vergleichsgrößen blieben Aussagen zur Höhe der Fehlermaße für Personen ohne ML–Erfahrung teilweise abstrakt. Zudem wurde in einem Durchgang eine vorangegangene Data–Leakage–Warnung nicht verstanden und daher ignoriert. Die Aufgabe spricht somit für eine weitgehende Zielerreichung auf der Ebene des Modellzugangs. Die eigenständige fachliche Bewertung der Modellgüte blieb jedoch von der Qualität und Tiefe der Systemerklärung abhängig.

Hinweise und Warnungen. Die Verständlichkeit und Entscheidungssicherheit bei der Bearbeitung von Warnungen erreichten jeweils einen Mittelwert von 4,67. Die Teilnehmenden erkannten grundsätzlich, dass bestimmte Verarbeitungsschritte vor der Trainings- und Testaufteilung ein Data–Leakage–Risiko erzeugen können. Die Warnfunktion stellte damit einen direkten Bezug zwischen der aktuellen Pipeline und einer möglichen methodischen Fehlerquelle her.

Die qualitativen Rückmeldungen zeigen jedoch deutliche Unterschiede in der tatsächlichen Interpretation. Eine Person beschrieb die Warnung als verständlich und hilfreich für die Einschätzung des eigenen Risikos. Eine andere Person konnte die Warnung aufgrund der Formulierung und grammatikalischer Unklarheiten nicht vollständig nachvollziehen. Zusätzlich führte der erneute Verweis auf bereits bearbeitete Schritte zu Verwirrung. In diesem Fall wurde die Warnung zunächst ignoriert, obwohl die Person davon ausging, sie durch eine weitere Nachfrage verstehen zu können.

Die Bewertung verdeutlicht, dass Warnungen im System sichtbar und grundsätzlich wahrnehmbar waren. Ihre methodische Wirkung hing jedoch von sprachlicher Präzision, Aktualität und Bezug zum tatsächlich aktiven PipelineZustand ab. Für Nutzerinnen und Nutzer ohne ML-Erfahrung müssen Warnungen nicht nur auf ein Risiko hinweisen, sondern

eindeutig erklären, welcher bestehende Schritt problematisch ist, warum das Problem entsteht und welche konkrete Korrektur möglich ist.

Versionierung und Variantenbildung. Die Versionierungsfunktion wurde von allen drei Teilnehmenden verwendet. Die Verständlichkeit und Entscheidungssicherheit lagen jeweils bei 4,33. Die Teilnehmenden konnten zu einem früheren Pipeline–Zustand zurückkehren und von dort eine alternative Verarbeitung oder ein anderes Modell ausprobieren. In einem Durchgang wurde beispielsweise zum Zustand vor dem Modelltraining zurückgekehrt und anschließend eine lineare Regression anstelle des zuvor verwendeten Modells gewählt.

Das Orientierungsgefühl fiel mit 3,67 niedriger aus. Die Freitextantworten zeigen, dass der Zweck des Zustandswechsels und die daraus resultierende Veränderung grundsätzlich nachvollziehbar waren. Schwierigkeiten entstanden jedoch bei der Auswahl des konkreten Zustands in der Oberfläche. Eine Person benötigte mehrere Versuche, um den gewünschten Rücksprungpunkt zu identifizieren. Auch wurde angemerkt, dass die Übersichtlichkeit bei einem längeren oder stärker verzweigten Verlauf abnehmen könnte.

Die Versionierung unterstützt damit das kontrollierte Experimentieren mit unterschiedlichen Pipeline–Varianten. Die Ergebnisse zeigen jedoch, dass eine funktionierende Rücksprungmöglichkeit nicht automatisch eine ausreichende visuelle Orientierung über Versionen, Unterschiede und Verzweigungen gewährleistet.

Zusammenfassende Bewertung der Aufgabenbearbeitung. Über die sechs Aufgaben hinweg konnten sämtliche Teilnehmenden zentrale Schritte eines tabellarischen ML-Workflows über natürliche Sprache anstoßen und durchführen. Besonders tragfähig war die Unterstützung bei der Datensatzerkundung, der Behandlung fehlender Werte, dem Modelltraining und der kontrollierten Rückkehr zu früheren Zuständen. Die Teilnehmenden mussten weder Programmcode erstellen noch die technische Implementierung der Verarbeitungsschritte kennen.

Schwierigkeiten traten insbesondere dann auf, wenn mehrere fachlich plausible Alternativen bewertet, Auswirkungen auf spätere Pipeline-Schritte abgeschätzt oder Warnungen auf bereits ausgeführte Verarbeitungsschritte bezogen werden mussten. Die Ergebnisse sprechen daher für eine weitgehende Zielerreichung auf der Ebene des technischen Zugangs und der natürlichsprachlichen Bedienung. Eine durchgehend eigenständige und fachlich informierte Bearbeitung konnte für komplexere Entscheidungssituationen dagegen nicht nachgewiesen werden.

Nach Abschluss der Aufgaben wurde die Nutzung anhand von zwölf Aussagen auf einer zehnstufigen Zustimmungsskala bewertet. Die Aussagen beziehen sich auf die Dimensionen

Verständlichkeit, Bedienbarkeit, Kontrolle und Gesamteindruck. Für sämtliche Aussagen liegen Bewertungen aller drei Teilnehmenden vor.

Tabelle 5.8: Durchschnittliche Bewertungen der Nachbefragung

Dimension	Aussage	Mittelwert
Verständlichkeit	Ich konnte nachvollziehen, was das System nach meinen Eingaben getan hat.	8,67
	Die Rückmeldungen des Systems waren verständlich.	7,33
	Die Warnungen des Systems waren verständlich.	7,67
Bedienbarkeit	Es fiel mir leicht, meine Absichten in natürlicher Sprache auszudrücken.	8,67
	Die Bedienung des Systems war intuitiv.	7,67
	Ich wusste meistens, wie ich die nächste Aufgabe lösen kann.	6,33
Kontrolle	Ich hatte jederzeit das Gefühl, die Kontrolle über Änderungen zu behalten.	9,00
	Die Bestätigungsmechanismen waren hilfreich.	9,33
	Die Möglichkeit zum Zurücksetzen auf frühere Zustände war hilfreich.	9,33
Gesamteindruck	Ich würde das System als verständlich bezeichnen.	9,00
	Ich würde das System auch ohne Programmierkenntnisse nutzen können.	7,00
	Das System erleichtert aus meiner Sicht den Zugang zu ML–Workflows.	8,00

Anmerkung: Bewertet wurde auf einer Skala von 1 bis 10. Höhere Werte entsprechen einer stärkeren Zustimmung. Allen Mittelwerten liegen drei Bewertungen zugrunde.

Die Nachbefragung bestätigt zunächst die grundsätzlich hohe wahrgenommene Verständlichkeit. Die Nachvollziehbarkeit der Systemhandlungen erreichte einen Mittelwert von 8,67. Die allgemeine Verständlichkeit des Systems wurde mit 9,00 bewertet. Die Rückmeldungen und Warnungen erhielten mit 7,33 beziehungsweise 7,67 niedrigere Werte. Diese Differenz stimmt mit den aufgabenbezogenen Freitextantworten überein. Das Systemverhalten war in seiner Gesamtheit nachvollziehbar, einzelne Erklärungen und Warntexte waren jedoch nicht in jedem Fall sprachlich präzise oder ausreichend auf den aktuellen Pipeline–Zustand bezogen. Der Mittelwert der Dimension Verständlichkeit beträgt 7,89.

Die Bedienbarkeit wurde ebenfalls positiv eingeschätzt. Das Formulieren eigener Absichten in natürlicher Sprache erreichte einen Mittelwert von 8,67. Die Intuitivität der Bedienung wurde mit 7,67 bewertet. Deutlich niedriger fiel die Klarheit über den jeweils

nächsten Schritt mit 6,33 aus. Dieses Ergebnis bestätigt den Befund aus Aufgabe 3. Das System konnte die Absichten der Teilnehmenden überwiegend verarbeiten, führte sie aber nicht durchgehend transparent durch die fachliche Entscheidungsfolge. Insbesondere bei mehreren möglichen Vorverarbeitungsschritten mussten zusätzliche Rückfragen gestellt werden. Der Mittelwert der Dimension Bedienbarkeit beträgt 7,56.

Die Dimension Kontrolle weist mit 9,22 die höchste zusammengefasste Bewertung auf. Das allgemeine Kontrollgefühl erreichte 9,00. Die Bestätigungsmechanismen und die Rücksprungmöglichkeit wurden jeweils mit 9,33 bewertet. In den Abschlussfragen wurde die explizite Bestätigung vor der Ausführung als besonders hilfreich beschrieben, da sie einen bewussten menschlichen Entscheidungsschritt zwischen Systemvorschlag und Ausführung einfügt. Damit stützt die Nachbefragung die Architekturentscheidung, zustandsverändernde Operationen kontrollierbar zu gestalten und frühere Pipeline-Zustände zu erhalten.

Die Bewertungen zum Gesamteindruck fallen ebenfalls positiv aus. Die Nutzbarkeit ohne Programmierkenntnisse erreichte einen Mittelwert von 7,00. Der erleichterte Zugang zu ML-Workflows wurde mit 8,00 bewertet. Die Teilnehmenden beschrieben, dass sie ohne das System nicht oder nur mit erheblich höherem Zeitaufwand von der Datensichtung bis zum trainierten Modell gelangt wären. Gleichzeitig zeigt der niedrigere Wert zur Nutzbarkeit ohne Programmierkenntnisse, dass die natürliche Sprachsteuerung fachliche Unsicherheiten nicht vollständig aufhebt. Der Mittelwert der Dimension Gesamteindruck beträgt 8,00.

Die offenen Abschlussfragen ergänzen diese Werte. Als besonders hilfreich wurden die Bereitstellung einer vollständigen Arbeitsumgebung, die ausführlichen Ergebnisdarstellungen, die Übersichtsansicht und die Bestätigungsmechanismen genannt. Kritik betraf wiederholte oder nicht mehr zum aktuellen Zustand passende Hinweise, die teilweise unmittelbare Ausführung von Schritten nach einer informationsbezogenen Nachfrage, die Auswahl eines konkreten Rollback-Zustands und das notwendige manuelle Scrollen zu neuen Systemausgaben. Mehrere Teilnehmende gaben an, den Systemempfehlungen stark gefolgt zu sein, obwohl grundsätzlich alternative Wege und weitere Erklärungen verfügbar waren.

Insgesamt stützen die Nachbefragung und die aufgabenbezogenen Ergebnisse die Annahme, dass Personen mit geringen oder fehlenden Vorkenntnissen wesentliche Schritte eines ML-Workflows mithilfe von Gysi bearbeiten können. Das zentrale Ziel eines natürlichsprachlich steuerbaren und kontrollierten Assistenzsystems wurde damit grundsätzlich erreicht. Die Ergebnisse zeigen zugleich, dass erfolgreiche Systemhandlungen nicht mit einem vollständigen Verständnis ihrer methodischen Konsequenzen gleichgesetzt werden dürfen. Insbesondere bei der Auswahl mehrerer Vorverarbeitungsschritte, der Inter-

pretation kontextabhängiger Warnungen und der Orientierung zwischen verschiedenen Pipeline-Zuständen besteht weiterer Unterstützungsbedarf.

5.3.5 Limitationen

Die Aussagekraft der Nutzerstudie ist durch die Stichprobe von drei Personen begrenzt. Die Ergebnisse erlauben keine statistische Generalisierung auf die gesamte Zielgruppe. Sie zeigen ausschließlich Tendenzen und konkrete Schwierigkeiten innerhalb der beobachteten Sitzungen. Durch die geringe Fallzahl wirken sich einzelne Bewertungen unmittelbar auf die berechneten Mittelwerte aus.

Eine weitere Einschränkung betrifft die subjektive Erhebung der aufgabenbezogenen Bewertungen. Die Skalen erfassen die wahrgenommene Verständlichkeit, Entscheidungssicherheit und Orientierung, nicht die fachliche Richtigkeit der getroffenen Entscheidungen. Hohe Bewertungen können daher auch dann auftreten, wenn eine Person einer Systemempfehlung folgt, ohne deren methodische Konsequenzen vollständig zu verstehen. Dies zeigt sich insbesondere in Aufgabe 3, bei der eine hohe Entscheidungssicherheit mit einer vergleichsweise niedrigen Verständlichkeitsbewertung zusammentraf.

Bei der Verständlichkeitsbewertung von Aufgabe 4 enthielt ein Erhebungsbogen eine doppelte Markierung am unteren und oberen Skalenende. Für die deskriptive Berechnung wurde auf Grundlage der zugehörigen positiven Freitextantwort die Markierung am oberen Skalenende als intendierte Angabe interpretiert. Diese nachträgliche Zuordnung ist mit Unsicherheit verbunden und begrenzt die Eindeutigkeit des entsprechenden Mittelwerts.

Die Sitzungen wurden von mindestens einer und in einzelnen Fällen von zwei Personen passiv beobachtet. Die Beobachtenden griffen während der regulären Aufgabenbearbeitung nicht ein. Eine Unterstützung war ausschließlich für technische Notfälle vorgesehen. Dennoch kann die Anwesenheit weiterer Personen das Antwortverhalten, die wahrgenommene Sicherheit oder die Bereitschaft zum selbstständigen Experimentieren beeinflusst haben.

Alle Teilnehmenden bearbeiteten die Aufgaben in derselben festgelegten Reihenfolge. Da die Aufgaben aufeinander aufbauten, konnten Unsicherheiten oder methodisch ungünstige Entscheidungen aus früheren Aufgaben die spätere Bearbeitung beeinflussen. Die Bewertungen der Aufgaben sind daher nicht vollständig unabhängig voneinander. Zudem erfolgte kein Vergleich mit einer Kontrollgruppe oder einem alternativen Werkzeug. Aus den Ergebnissen lässt sich deshalb nicht ableiten, ob die Aufgaben mit Gysi schneller, fachlich korrekter oder verständlicher bearbeitet wurden als mit anderen Systemen.

Die Freitextantworten zeigen außerdem, dass einzelne Teilnehmende Empfehlungen des Systems übernahmen, obwohl deren fachliche Auswirkungen nicht vollständig verstanden

wurden. Eine erfolgreiche Ausführung kann deshalb nicht in jedem Fall als Nachweis einer eigenständigen Entscheidung interpretiert werden. Dies betrifft insbesondere die Auswahl mehrerer Vorverarbeitungsschritte, den Umgang mit Data–Leakage–Warnungen und die Interpretation von Modellkennzahlen. Die Studie belegt somit erste Nutzbarkeitstendenzen, nicht jedoch eine verlässlich autonome Bearbeitung von ML-Workflows durch die gesamte Zielgruppe.

6 Diskussion, Fazit und Ausblick

6.1 Diskussion

Die Ergebnisse zeigen, dass das zentrale Ziel der Arbeit grundsätzlich erreicht wurde. Mit Gysi konnte ein Prototyp entwickelt werden, der wesentliche Schritte eines tabellarischen Machine-Learning-Workflows über natürliche Sprache steuerbar macht. Der End-to-End-Walkthrough belegt, dass sich Datenexploration, Vorverarbeitung, Modelltraining und Ergebnisdarstellung innerhalb eines zusammenhängenden Arbeitsablaufs durchführen lassen. Gleichzeitig bleibt die technische Ausführung durch die Trennung von LLM und Backend– kontrollierbar. Insbesondere die strukturierte Pipeline-Spezifikation, der Bestätigungsmechanismus sowie die Versionierung verhindern, dass natürlichsprachliche Eingaben unmittelbar zu unkontrollierten oder irreversiblen Änderungen führen.

Damit bestätigt sich die in Kapitel 3 vorgenommene Einordnung von Gysi zwischen offenen agentischen Systemen und stark begrenzten Workflow-Assistenten. Im Gegensatz zu Ansätzen wie Data Interpreter verzichtet Gysi bewusst auf freie Codegenerierung und weitreichende Autonomie. Diese Einschränkung reduziert zwar die Flexibilität des Systems, erhöht jedoch die Nachvollziehbarkeit und Reproduzierbarkeit der ausgeführten Schritte. Gegenüber einem reinen Workflow-Generator bietet Gysi zudem einen persistenten, interaktiv veränderbaren Pipeline-Zustand, der Rollbacks und alternative Modellierungsvarianten ermöglicht. Der Beitrag des Systems liegt somit weniger in einer möglichst autonomen Durchführung von Data Science, sondern in der kontrollierten Verbindung von natürlicher Sprache und einem klar definierten ML-Funktionsraum.

Die explorative Nutzerstudie stützt diese Bewertung. Besonders positiv wurden das Kontrollerleben, die Bestätigungsmechanismen und die Rückkehr zu früheren Zuständen bewertet. Die zusammengefasste Bewertung der Dimension Kontrolle erreichte 9,22 von 10 Punkten. Auch das Formulieren eigener Absichten in natürlicher Sprache wurde mit 8,67 Punkten positiv eingeschätzt. Dies spricht dafür, dass die gewählte Architektur nicht nur technisch nachvollziehbar ist, sondern auch aus Nutzersicht ein hohes Maß an Kontrolle vermitteln kann.

Gleichzeitig zeigen die Ergebnisse eine wichtige Grenze der natürlichsprachlichen Be-

dienung. Ein erleichterter technischer Zugang ist nicht mit einem vollständigen fachlichen Verständnis gleichzusetzen. Besonders deutlich wird dies bei der Auswahl weiterer Vorverarbeitungsschritte. Hier stand einer vergleichsweise niedrigen Verständlichkeitsbewertung von 3,33 eine hohe Entscheidungssicherheit von 4,67 gegenüber. Teilnehmende konnten somit Entscheidungen treffen und ausführen, ohne deren methodische Konsequenzen immer vollständig beurteilen zu können. Auch Data–Leakage–Warnungen und Modellkennzahlen wurden zwar grundsätzlich wahrgenommen, mussten jedoch präzise, kontextbezogen und handlungsorientiert erklärt werden, um tatsächlich wirksam zu sein.

Insgesamt verdeutlichen die Ergebnisse, dass Gysi die programmiertechnische Einstiegshürde zu ML-Workflows reduzieren kann, fachliche ML-Kompetenz jedoch nicht ersetzt. Für ein Assistenzsystem dieser Art reicht es deshalb nicht aus, korrekte Aktionen bereitzustellen und deren Ausführung abzusichern. Ebenso wichtig ist eine didaktische Unterstützung, die Alternativen, Auswirkungen und sinnvolle nächste Schritte verständlich einordnet. Die Ergebnisse zeigen damit, dass die Qualität eines konversationellen ML-Assistenten nicht allein von seinem Funktionsumfang, sondern ebenso von der verständlichen Vermittlung fachlicher Zusammenhänge abhängt.

6.2 Limitationen

Die Ergebnisse dieser Arbeit sind vor dem Hintergrund des prototypischen Entwicklungsumfangs zu betrachten. Ziel war nicht die Umsetzung eines vollständigen AutoML-Systems, sondern die Untersuchung, ob sich natürlichsprachliche Interaktion mit einer kontrollierten, deterministischen und nachvollziehbaren Ausführung von Machine–Learning–Pipelines verbinden lässt. Der Funktionsumfang wurde daher bewusst begrenzt. Diese Abgrenzung erfolgte in Abstimmung mit dem betreuenden Professor und ermöglichte eine vertiefte Umsetzung der zentralen Architekturprinzipien.

Der aktuelle Prototyp unterstützt nur einen definierten Katalog an Vorverarbeitungs–, Modellierungs– und Erklärungsfunktionen. Weiterführendes Feature Engineering, eine systematische Hyperparameteroptimierung sowie zusätzliche Modellklassen wurden nicht umgesetzt. Auch lokale Erklärungen einzelner Vorhersagen, etwa durch SHAP und SHAPE Functions, sind nicht enthalten. Die Explainability Funktionen konzentrieren sich auf globale Zusammenhänge, Feature Importances und vergleichende Modellmetriken. Dadurch bleibt die funktionale Breite des Systems begrenzt, auch wenn die realisierten Funktionen für den gewählten Anwendungskontext ausreichend sind.

Eine weitere Einschränkung ergibt sich aus der Ausrichtung auf den Bike–Sharing–Datensatz. Die Auswahl der Verarbeitungsschritte, Modelle und Erklärungsformen wurde

an dessen Struktur und Regressionsaufgabe angepasst. Die Systemarchitektur und die Pipeline Spezifikation sind grundsätzlich erweiterbar. Der aktuelle Prototyp wurde jedoch nicht für beliebige Datensätze validiert. Andere Datensätze können abweichende Datentypen, Zielvariablen oder Qualitätsprobleme enthalten, die durch den bestehenden Aktionskatalog nicht ausreichend abgedeckt werden. Die Arbeit zeigt daher die Machbarkeit des Ansatzes innerhalb eines kontrollierten Anwendungsszenarios, nicht dessen uneingeschränkte Generalisierbarkeit.

Auch die eingesetzte Sprachmodellkomponente begrenzt die Systemleistung. Die Modellauswahl stellt einen Kompromiss zwischen fachlicher Leistungsfähigkeit, Antwortzeit und laufenden API–Kosten dar. Da die Kosten im Rahmen der Arbeit selbst getragen wurden, wurde kein durchgehend leistungsstärkeres Modell eingesetzt. Ein leistungsfähigeres Modell könnte komplexe Eingaben und fachliche Rückfragen möglicherweise zuverlässiger verarbeiten, würde jedoch höhere Kosten verursachen und die grundsätzliche Unsicherheit generativer Ausgaben nicht beseitigen.

Trotz strukturierter Aktionsformate, Schema–Validierung und begrenzter Berechtigungen können unklare Erklärungen, redundante Vorschläge oder fehlerhafte Interpretationen der Nutzerabsicht auftreten. Die deterministische Backend–Ausführung verhindert zwar eine unkontrollierte Codeausführung, garantiert jedoch nicht, dass jede vorgeschlagene Handlung fachlich optimal oder für unerfahrene Nutzende ausreichend erklärt ist.

Das aktuelle Kontextmanagement übermittelt die Gesprächshistorie, den Pipeline–Zustand und weitere Metadaten an das Sprachmodell. Mit zunehmender Länge einer Session steigt dadurch die Zahl der verarbeiteten Token. Dies erhöht Kosten und Antwortzeit und kann langfristig an die Kontextgrenze des Modells führen. Pro Nutzernachricht können zudem ein oder mehrere sequenzielle Sprachmodellaufrufe erforderlich sein. Ein Batch–Verfahren oder eine automatische Verdichtung älterer Gesprächsteile wurde nicht umgesetzt. Für längere Sessions wären daher Mechanismen zur Zusammenfassung und Auswahl relevanter Kontextinformationen erforderlich.

Eine weitere Limitation betrifft die Perspektive der Entwicklung. Das System wurde von zwei Studierenden entwickelt, die mit Aufbau und Anforderungen von Machine–Learning–Pipelines vertraut sind. Die Zielgruppe besteht dagegen aus Personen mit geringen oder fehlenden Vorkenntnissen. Daraus kann eine Verzerrung bei der Einschätzung entstehen, welche Begriffe, Hinweise oder Abläufe für Endnutzende verständlich sind. Dies betrifft insbesondere Modellmetriken, Data–Leakage, die Reihenfolge von Pipeline–Schritten und die Auswahl zwischen mehreren Verfahren.

Der Prototyp wurde außerdem nicht über einen längeren Zeitraum durch Endnutzende eingesetzt. Die Evaluation bestand aus einzelnen aufgabenbasierten Sitzungen. Eine

iterative Erprobung mit mehreren aufeinanderfolgenden Entwicklungs- - und Testzyklen war im verfügbaren Zeitrahmen nicht möglich. Erkannte Probleme, etwa unklare Warnungen, redundante Hinweise, Schwierigkeiten bei der Auswahl eines Rollback-Zustands oder die Navigation zu neuen Chat– Ausgaben, konnten deshalb nicht erneut umgesetzt und anschließend systematisch überprüft werden.

Die empirische Aussagekraft der Nutzerstudie ist durch die Stichprobengröße von drei Personen begrenzt. Die geringe Fallzahl ergibt sich aus dem zeitlichen Rahmen und dem Aufwand der moderierten Durchführung. Die Ergebnisse zeigen erste Tendenzen, wiederkehrende Schwierigkeiten und mögliche Stärken des Systems, erlauben jedoch keine statistische Generalisierung auf die gesamte Zielgruppe. Eine weiterführende Studie sollte eine größere und heterogenere Stichprobe, längere Nutzungsszenarien und einen Vergleich mit alternativen Werkzeugen einbeziehen.

Insgesamt resultieren mehrere Limitationen aus bewusst gewählten Abgrenzungen. Die Konzentration auf einen Datensatz, einen begrenzten Funktionsumfang und eine kontrollierte Architektur ermöglichte eine detaillierte Untersuchung des zentralen Systemkonzepts. Gleichzeitig begrenzen diese Entscheidungen die funktionale Breite, die Übertragbarkeit und die empirische Aussagekraft. Die Arbeit ist daher als prototypischer Machbarkeitsnachweis und als Grundlage für weiterführende technische und empirische Untersuchungen einzuordnen.

6.3 Fazit und Ausblick

Die vorliegende Arbeit untersuchte, wie sich Machine-Learning-Pipelines über natürliche Sprache steuern lassen, ohne die Kontrolle über ihre technische Ausführung an ein Large Language Model abzugeben. Mit Gysi wurde ein prototypischer konversationeller ML-Assistent entwickelt, der natürlichsprachliche Eingaben in einen begrenzten und validierbaren Aktionsraum überführt. Die Arbeit zeigt damit für den gewählten Anwendungskontext, dass eine flexible sprachliche Interaktion mit einer kontrollierten und reproduzierbaren Backend-Ausführung verbunden werden kann.

Der entwickelte Prototyp ermöglicht einen zusammenhängenden Arbeitsablauf von der Datensatzerkundung über die Vorverarbeitung bis zum Modelltraining und zur Ergebnisinterpretation. Die praktische Erprobung und die explorative Nutzerstudie liefern erste Hinweise darauf, dass dieser Ansatz den Zugang zu ML-Workflows für Personen ohne Programmierkenntnisse erleichtern kann. Gleichzeitig bleibt die fachliche Beurteilung der vorgeschlagenen Verarbeitungsschritte eine zentrale Aufgabe der Nutzer. Der wesentliche Beitrag der Arbeit liegt daher nicht in der Automatisierung fachlicher Entscheidungen,

sondern in der Entwicklung einer kontrollierten Schnittstelle, über die solche Entscheidungen verständlich vorbereitet und technisch nachvollziehbar umgesetzt werden können.

Ausgehend von diesem Machbarkeitsnachweis ergeben sich mehrere Ansatzpunkte für zukünftige Arbeiten. Zunächst sollte untersucht werden, inwieweit sich das System auf weitere Datensätze und Aufgabenstellungen übertragen lässt. Der bestehende Prototyp ist auf einen tabellarischen Regressionsdatensatz zugeschnitten. Eine Erweiterung auf Klassifikationsprobleme, unterschiedliche Datenstrukturen und zusätzliche Modellklassen würde zeigen, ob sich die entwickelte Architektur auch außerhalb des bisherigen Anwendungskontexts bewährt. Durch die modulare Step Registry und die Pipeline-Spezifikation können neue Verarbeitungsschritte grundsätzlich ergänzt werden, ohne die zentrale Schnittstelle zwischen LLM und Backend neu gestalten zu müssen.

Auch der fachliche Funktionsumfang könnte erweitert werden. Denkbar sind zusätzliche Verfahren des Feature Engineerings, eine automatisierte Hyperparameteroptimierung sowie lokale Erklärungen einzelner Vorhersagen. Solche Erweiterungen sollten weiterhin innerhalb klar definierter und validierbarer Backend-Funktionen umgesetzt werden. Auf diese Weise ließe sich der Funktionsumfang erhöhen, ohne das grundlegende Kontrollprinzip des Systems aufzugeben.

Ein weiterer Entwicklungsschwerpunkt betrifft die Qualität und Zielgruppengerechtigkeit der Erklärungen. Zukünftige Versionen könnten die bereitgestellten Informationen stärker an den Kenntnisstand und die konkrete Entscheidungssituation der Nutzer anpassen. Bei mehreren fachlich plausiblen Optionen sollte das System deren Voraussetzungen, Vor- und Nachteile sowie Auswirkungen auf nachgelagerte Pipeline-Schritte vergleichend darstellen. Auch Modellkennzahlen könnten durch Referenzmodelle, anschauliche Beispiele oder zusätzliche Visualisierungen besser eingeordnet werden. Darüber hinaus sollte die Unterscheidung zwischen informationsbezogenen Nachfragen und tatsächlichen Ausführungsabsichten zuverlässiger erfolgen, damit eine Bitte um Erklärung nicht unbeabsichtigt als Handlungsauftrag interpretiert wird.

Für längere Nutzungssitzungen wäre zudem ein effizienteres Kontextmanagement sinnvoll. Ältere Gesprächsinhalte könnten automatisch verdichtet und nur bei Bedarf erneut in den Modellkontext aufgenommen werden. Dies könnte den Tokenverbrauch, die laufenden Kosten und die Antwortzeiten reduzieren. Eine Spracheingabe über Speech-to-Text stellt ebenfalls eine mögliche Erweiterung dar, sollte jedoch erst auf Grundlage einer robusten textbasierten Interaktion umgesetzt werden.

Schließlich sollte die Evaluation mit einer größeren und heterogeneren Stichprobe fortgeführt werden. Neben subjektiven Bewertungen sollten zukünftige Studien auch die fachliche Korrektheit der Entscheidungen, die Bearbeitungszeit und die Häufigkeit problema-

tischer Pipeline-Konfigurationen erfassen. Ein Vergleich mit codebasierten Werkzeugen, grafischen Benutzeroberflächen oder bestehenden AutoML-Systemen könnte zeigen, bei welchen Aufgaben und Nutzergruppen der konversationelle Ansatz konkrete Vorteile bietet. Auf dieser Grundlage könnte Gysi schrittweise von einem prototypischen Machbarkeitsnachweis zu einem breiter einsetzbaren Assistenzsystem für kontrollierte Machine-Learning-Workflows weiterentwickelt werden.

Quellenverzeichnis

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C. & Amodei, D. (2020): Language Models are Few-Shot Learners.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Ponde, H., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S. & Zaremba, W. (2021): Evaluating Large Language Models Trained on Code.

Döbel, I., Leis, M., Molina Vogelsang, M., Neustroev, D., Petzka, H., Riemer, A., Rüping, S., Voß, A., Wegele, M. & Welz, J. (2018): Maschinelles Lernen. Eine Analyse zu Kompetenzen, Forschung und Anwendung. URL: `https://publica.fraunhofer.de/handle/publica/299081`.

Grinsztajn, L., Oyallon, E. & Varoquaux, G. (2022): Why do tree-based models still outperform deep learning on tabular data? URL: `https://arxiv.org/abs/2207.08815`.

Gupta, N., Gschwind, T., Chakraborty, S., Mehta, S., Tyler, T., Sisodia, S. & Clermont, B. (2026): From Natural Language to Executable ETL Flows: The IBM DataStage Assistant. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40, S. 39949–39957. URL: `https://doi.org/10.1609/aaai.v40i47.41429`.

Hong, S., Lin, Y., Liu, B., Liu, B., Wu, B., Zhang, C., Li, D., Chen, J., Zhang, J., Wang, J., Zhang, L., Zhang, L., Yang, M., Zhuge, M., Guo, T., Zhou, T., Tao, W., Tang, R., Lu, X., Zheng, X., Liang, X., Fei, Y., Cheng, Y., Ni, Y., Gou, Z., Xu, Z., Luo, Y. & Wu, C. (2025): Data Interpreter: An LLM Agent for Data Science. In: *Findings of the Association for Computational Linguistics: ACL 2025*. Hrsg. von W. Che, J. Nabende, E. Shutova & M. T. Pilehvar. Vienna, Austria: Association for Computational

Linguistics, S. 19796–19821. ISBN: 979-8-89176-256-5. URL: https://aclanthology.org/2025.findings-acl.1016/.

Maynez, J., Narayan, S., Bohnet, B. & McDonald, R. (2020): On Faithfulness and Factuality in Abstractive Summarization. In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Hrsg. von D. Jurafsky, J. Chai, N. Schluter & J. Tetreault. Online: Association for Computational Linguistics, S. 1906–1919. URL: https://aclanthology.org/2020.acl-main.173/.

McKinney, W. (2010): Data Structures for Statistical Computing in Python. In: *SciPy 2010*, URL: https://doi.org/10.25080/Majora-92bf1922-00a.

OpenAI (2025a): GPT-4.1 Mini Model. OpenAI. URL: https://platform.openai.com/docs/models/gpt-4-1-mini (besucht am 13. 06. 2026).

OpenAI (2025b): Introducing GPT-4.1 in the API. OpenAI. URL: https://openai.com/index/gpt-4-1/ (besucht am 13. 06. 2026).

OpenAI (2025c): Models. OpenAI. URL: https://platform.openai.com/docs/models (besucht am 13. 06. 2026).

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Müller, A., Nothman, J., Louppe, G., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. & Duchesnay, É. (2018): Scikit-learn: Machine Learning in Python. URL: https://arxiv.org/abs/1201.0490.

Thornton, C., Hutter, F., Hoos, H. H. & Leyton-Brown, K. (2013): Auto-WEKA: combined selection and hyperparameter optimization of classification algorithms. In: *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD '13. Chicago, Illinois, USA: Association for Computing Machinery, S. 847–855. ISBN: 9781450321747. URL: https://doi.org/10.1145/2487575.2487629.

Xu, F. F., Vasilescu, B. & Neubig, G. (2021): In-IDE Code Generation from Natural Language: Promise and Challenges. URL: https://arxiv.org/abs/2101.11149.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. & Cao, Y. (2023): ReAct: Synergizing Reasoning and Acting in Language Models. URL: https://arxiv.org/abs/2210.03629.

Yin, P., Li, W.-D., Xiao, K., Rao, A., Wen, Y., Shi, K., Howland, J., Bailey, P., Catasta, M., Michalewski, H., Polozov, O. & Sutton, C. (2023): Natural Language to Code Generation in Interactive Data Science Notebooks. In: S. 126–173.